Digital Bluetooth Stethoscope

The 1st meeting

Presenter:Xu Ruijing (Rae)

Date:8th,October,2025

OVERVIEW

1 Introduction

2 Literature Review

3 Existing Products

4 Future Work

Background

World Health Organization *(World Health Organization, 2025)*

Cardiovascular diseases (CVDs)心血管疾病 are the leading cause of death globally.

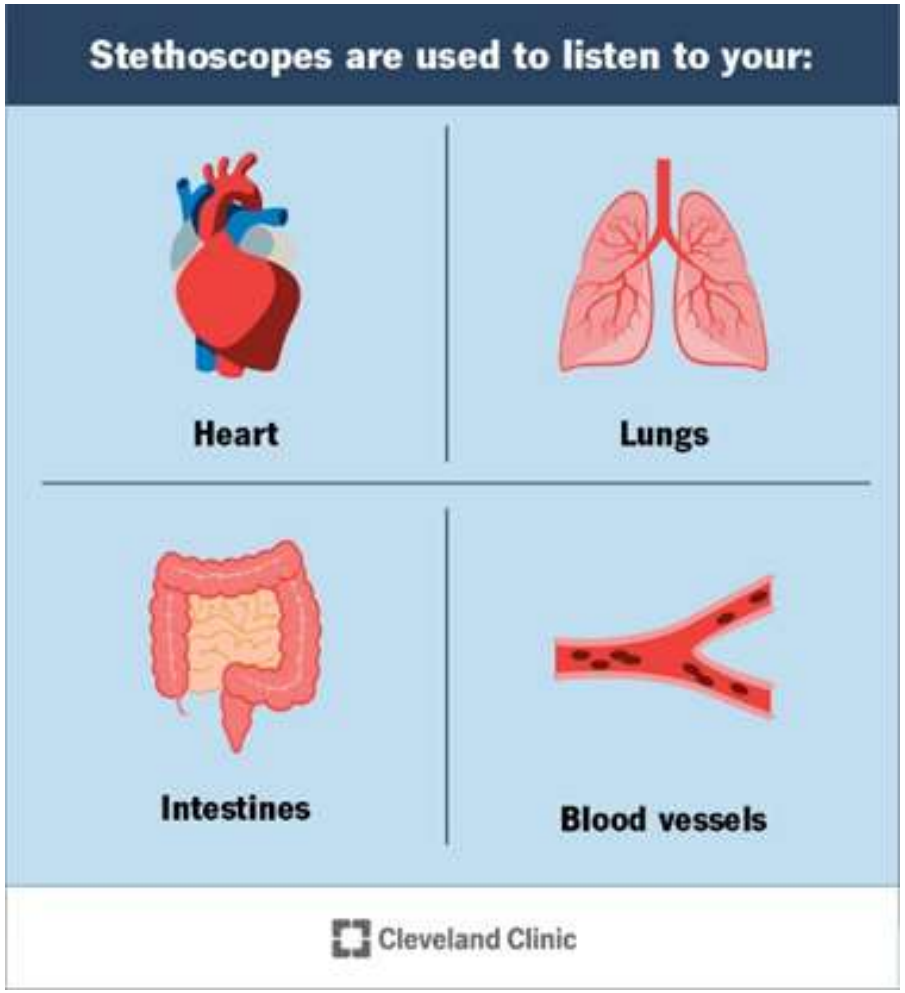
- An estimated **19.8 million** people died from CVDs in 2022, representing approximately **32%** of all global deaths.
- **Over three quarters** of CVD deaths take place in low- and middle-income countries. → Lack of primary health care programmes

Singapore Heart Foundation
Your Heart We Care

In Singapore, about **22** people die from cardiovascular disease every day. It accounted for **30.5%** of all deaths in 2024.*(My Heart (Singapore), n.d.)*

World Health Organization. (2025, July 31). Cardiovascular diseases (CVDs). https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-(cvds)
MyHeart. (n.d.). *Heart disease statistics*. https://www.myheart.org.sg/health/heart-disease-statistics/

Medical Knowledge



Cardiac Auscultation 心脏听诊 (*Harrison's principles of internal medicine* (22nd ed.))

1. **Heart Sounds心音**
   a) The first heart sound S1 (Closure of mitral and tricuspid valves 二尖瓣和三尖瓣关闭)
   b) The second heart sound S2 (Closure of aortic (A2) and pulmonic (P2) valves 主动脉瓣（A2）和肺动脉瓣（P2）关闭)
2. **Murmurs杂音** (Systolic Murmurs收缩期、Diastolic Murmurs舒张期、Continuous Murmurs连续杂音)
3. **Additional Heart Sounds：S3、S4**

heart failure during diastolic period. In general, the frequency of S1 is lower than that of S2, and the duration of S1 is longer than that of S2. The S3 occurs from 0.1 to 0.2 s after S2, while S4 occurs from 0.07 to 0.1 s before S1—both of them are low pitched. In addition to

Key points:

- high selective sensitivity
- corrupted with noise
- subjective interpretation

Medical Knowledge

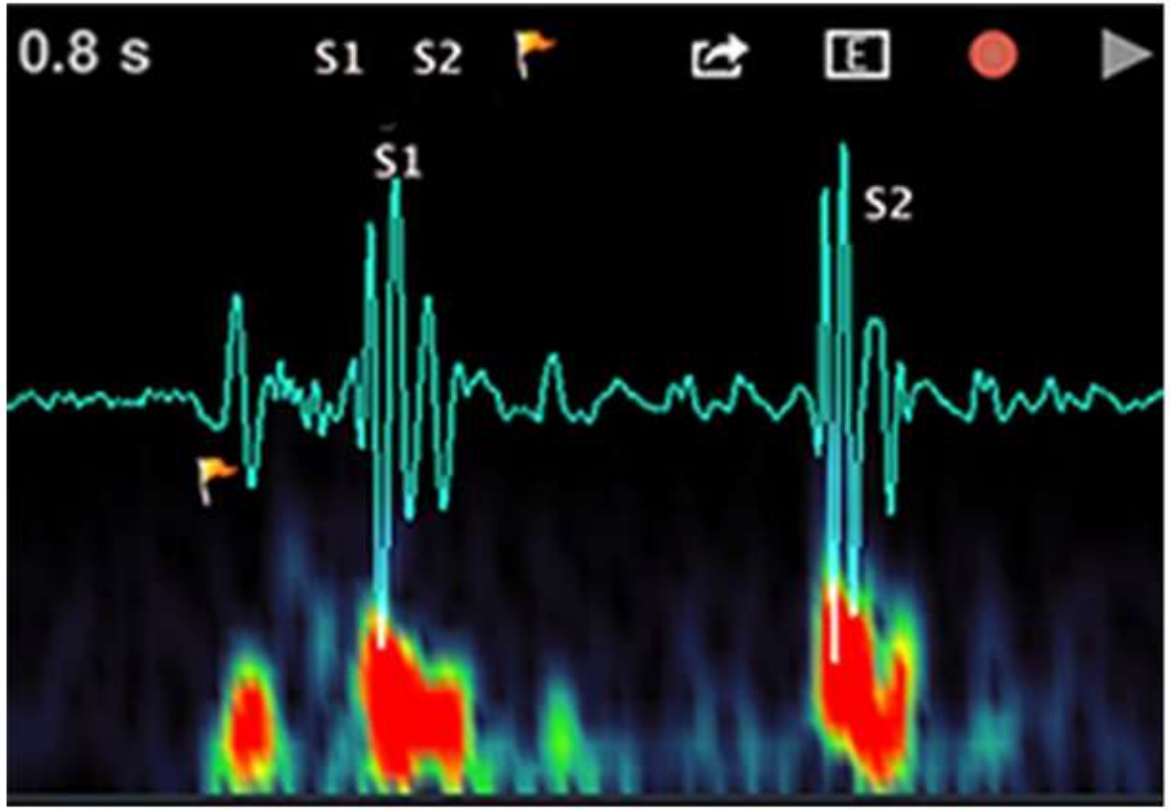


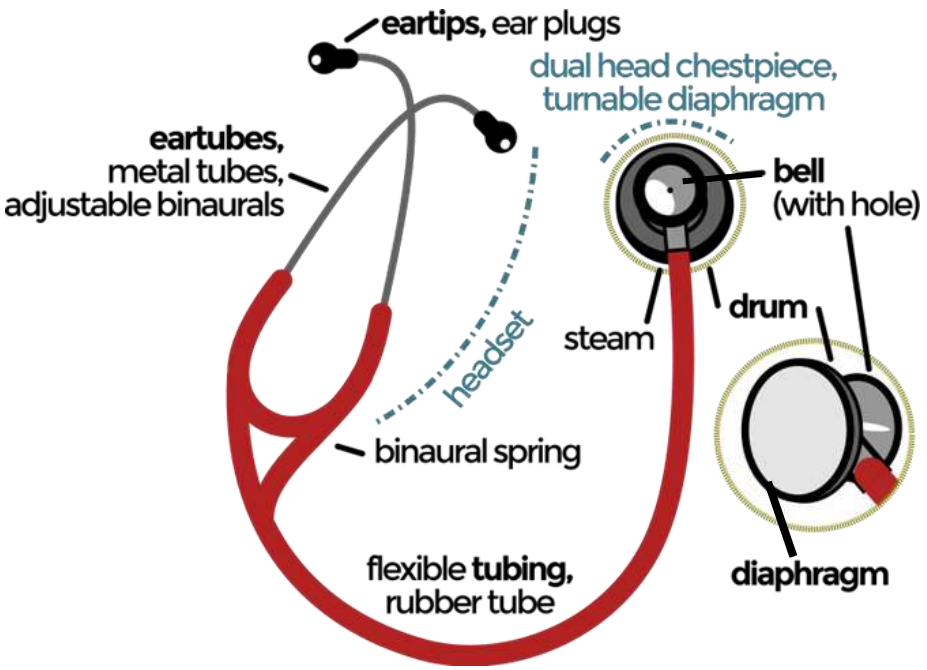
Figure 1 Two primary heart sounds: S1 and S2.

Leng, S., Tan, R. S., Chai, K. T., Wang, C., Ghista, D., & Zhong, L. (2015). The electronic stethoscope. *Biomedical engineering online*, *14*, 66. https://doi.org/10.1186/s12938-015-0056-y

Stethoscope

A stethoscope is an **acoustic transmission device** that uses **air/solid media conduction and the principle of resonance.** 空气/固体介质传声和共振原理

The stethoscope does not "amplify" the sound energy itself, but enhances the recognition of useful sounds by blocking ambient noise, matching acoustic impedance, and selectively transmitting certain frequencies.



Non-invasive, low-cost, and portable

Chestpiece 胸件/听诊头
= bell (low-frequency)
+ diaphragm隔膜 (high-frequency)

Review on Stethoscope

Analogue 模拟(传统式)

- Consists of a **chest piece (bell + diaphragm)**, **hollow tubing**, and **binaural earpiece**
- Purely mechanical amplification without electronics
- Simple, low cost, easy

➢ **3M Littmann Classic III (100 US dollars)**

Electronic 电子

- using a **microphone or piezoelectric sensor**.
- Signals are **amplified and filtered.**
- background noise and electromagnetic interference.

➢ Limited features, poor value, low demand

Digital 数字

- **digital signal processing (DSP)**, **Bluetooth transmission**, and **AI analysis**.
- Sounds are **digitized and visualized** (waveforms, spectrograms).
- AI algorithms have limits

➢ **Eko Core 500**

Seah, J. J., Zhao, J., Wang, D. Y., & Lee, H. P. (2023). Review on the Advancements of Stethoscope Types in Chest Auscultation. Diagnostics, 13(9), 1545. https://doi.org/10.3390/diagnostics13091545

Leng, S., Tan, R. S., Chai, K. T., Wang, C., Ghista, D., & Zhong, L. (2015). The electronic stethoscope. *Biomedical engineering online*, *14*, 66. https://doi.org/10.1186/s12938-015-0056-y

[17]. For the data acquisition module, an electronic stethoscope records the HS and the associated electronics converts it into digital signals and sends it to the pre-processing module. In the pre-processing module, the filtered and interference reduced HS signal is normalized and segmented. Feature extraction and classification are carried out in the signal processing module. The output of the system is the classification result for clinical diagnostic decision making. Detailed descriptions on the three main modules and their sub-modules are provided as follows.

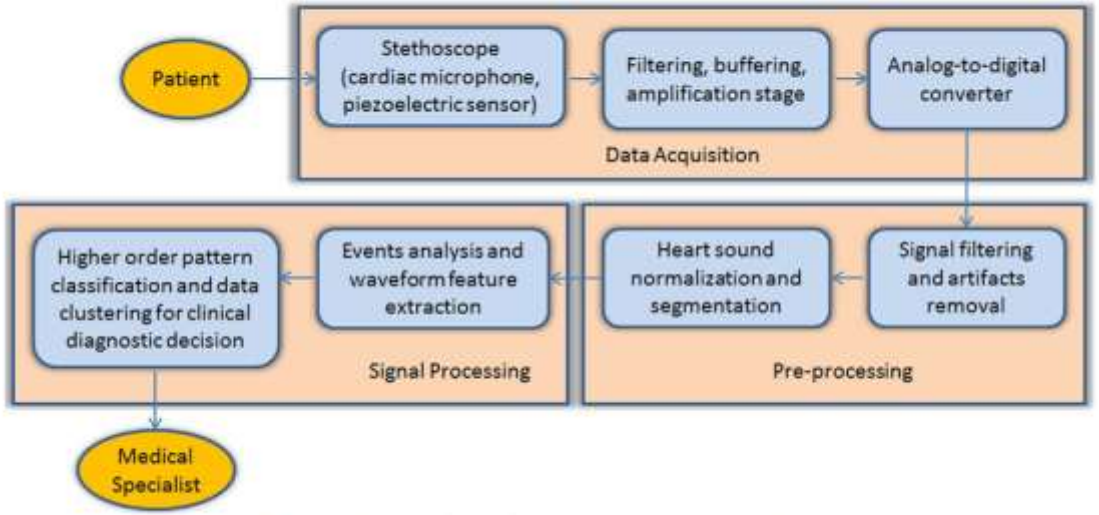


Figure 4 Typical flow chart for heart sound signal acquisition, processing and analysis.

1. Heart Sound Data Acquisition Module

Main components: Sensor(microphone, piezoelectric, MEMS capacitive), Amplifier, Filter (Band-Pass), ADC
selective sensitivity: typically < 20 mV and between 20 Hz–2000 Hz.

2. Pre-processing Module

Denoising, Normalization, Segmentation
non-stationary心音信号非平稳性: heart rate, noise, and respiration all vary with time.

3. Signal Processing Module

Feature Extraction, Classification(ML algorithms)

Leng, S., Tan, R. S., Chai, K. T., Wang, C., Ghista, D., & Zhong, L. (2015). The electronic stethoscope. *Biomedical engineering online*, *14*, 66. https://doi.org/10.1186/s12938-015-0056-y

Pre-processing Module → **Denosing Methods**

- **Time Domian Denoising**

Time domain denoising An adaptive noise canceller (ANC) is employed for external noise reduction [33, 43, 44]. The ANC, as shown in Figure 7, consists of two sound transducers. The first one, called primary input, is used to pick up the noise corrupted HS. The second one, named reference input, is used to only pick up the environmental noise correlated in some unknown way with the primary noise. The reference input is filtered and subtracted from the primary input to obtain the signal estimate.

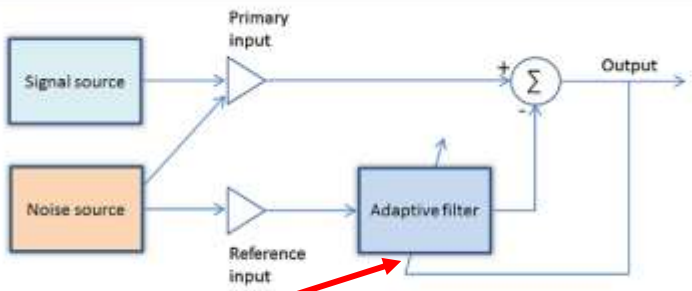


Figure 7 The structure of adaptive noise canceller.

LMS最小均方, RLS递归最小二乘

- Drawback: require **two** sensors → SIANC单通道

generates a reference signal using the primary signal.

the adaptive line enhancer (ALE) is also used for denoising the HS. the reference signal in ALE is the delayed version of the primary signal.

Single sensor, single channel

Working Mechanism:

Heartbeat signals exhibit periodic characteristics, whereas noise (such as breathing or speech sounds) are not.

The filter adaptively **enhances components while suppressing non-periodic noise.**

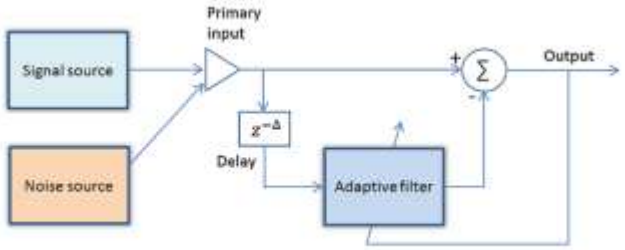


Figure 8 The structure of adaptive line enhancer.

LMS-ALE, RLS-ALE(fast convergence and small steady-state error, but with greater computational complexity)

Leng, S., Tan, R. S., Chai, K. T., Wang, C., Ghista, D., & Zhong, L. (2015). The electronic stethoscope. *Biomedical engineering online*, *14*, 66. https://doi.org/10.1186/s12938-015-0056-y

Pre-processing Module → Denosing Methods

- **Transform Domain Denosing**

ing it from the time domain into the frequency domain. The FT is usually implemented in the form of the fast FT algorithm (FFT). One drawback of FT is that it is not suited for analyzing non-stationary signals because it does not reveal how the signal's frequency contents vary with time. On the other hand, as most real world signals are generally non-stationary in nature (such as HS signals), a new signal processing technique that is able to handle the non-stationarity of a signal is needed. The short-time FT (STFT) is one solution to overcome the limitations of the FT [46]. The STFT analyzes a small section of the (stationary or non-stationary) signal at a time, which is known as windowing. The time domain signal is decomposed into a 2D time–frequency representation using STFT, and the variations of the frequency content of that signal within the window function are revealed. The problem with STFT is a compromise in resolution. A wide window gives better frequency resolution but poor time resolution. A narrow window results in good time resolution but poor frequency resolution. This resolution problem of STFT is solved by wavelet transform (WT) [47] which uses variable-length window. It was developed as a method to obtain high resolution in both time and frequency. A WT in which the wavelets are discretely sampled is known as the discrete WT (DWT).

The general wavelet denoising procedure of DWT are:
decomposition, thresholding, reconstruction

Besides WT, the empirical mode decomposition (EMD) [59], as another method for denoising, has become rather popular. The EMD process is a way to decompose a signal into so-called intrinsic mode functions (IMF), and obtain instantaneous frequency data. It has proven to be quite versatile in a broad range of applications for extracting signals from data generated in noisy non-linear and non-stationary processes. Mode mixing is one of the drawbacks of the original EMD, wherein a single IMF may consist of signals of widely disparate scales or a signal of similar scales may reside in different IMFs. In order to overcome this problem, the Ensemble EMD (EEMD) constitutes a noise-assisted data approach proposed in [60]. This algorithm defines IMF components as an ensemble of trials, each consisting of original signals contaminated with white noise of finite amplitude. A number of denoising methods of HS using EMD or EEMD can be found in [41,

- EMD decomposes a signal into multiple intrinsic modal functions (IMFs)内在模态函数,
- EEMD reduces "modal aliasing" by adding white noise and averaging the results.
- For nonlinear, non-stationary signals , EEMD's is better.

Eko Core 500 (website: https://www.ekohealth.com/products/core-500-digital-stethoscope?variant=39999867879520)

Eko Core 500

Features

- TrueSound™ technology with in-ear speakers
- Full-color LCD display
- 3-lead ECG with frequency response range from 0.1-250 Hz

The heart has four regions, with each recording lasting 30 seconds, so just listening to the heart takes two minutes.

- Up to 40x amplification
- Active noise cancellation
- Bluetooth wireless listening option

Bluetooth 4.2

Dimensions & Weight

- 27 in (69 cm) L from ear tip to chestpiece
- 6.6 oz (186 gm) ← Heavier than traditional ones (about 100gm)
- Chestpiece weight: 3.1 oz (88 gm)
- Diaphragm diameter: 1.2 in (3.0 cm)

APP:
Eko: Digital Stethoscopes

STEP 1

Conduct exam as usual.

STEP 2

AI analyzes as you listen.

STEP 3

Insights, instantly.

Crucial heart disease indicators — murmurs, AFib, and low ejection fraction — are **flagged in 15 seconds**, giving you

STEP 4

Save, track, and share.

Eko Core 500's Drawbacks

- 6.6 oz (186 gm) ← Heavier than traditional ones (about 100gm)
  - Use smaller battery capacity whenever possible (related to battery life)
    - Bluetooth consumes less power

- "It amplifies noise from fabric/skin friction as well which is annoying."（会把衣物摩擦或皮肤摩擦的噪声也放大，比较烦人）
  - Recognition of background noise requires improvement

- Regulatory and usage restrictions
  - FDA certification is present but insufficient
  - Patient privacy
  - Diagnostic fees

In the Following Two Weeks…

- Review relevant literature to gain an in-depth understanding of the **operational mechanisms of each module**, as well as the **data processing algorithms**.
- Considering potential **electronic components**.
- Learning viable **AI models** for diagnostic applications.

Digital Bluetooth Stethoscope

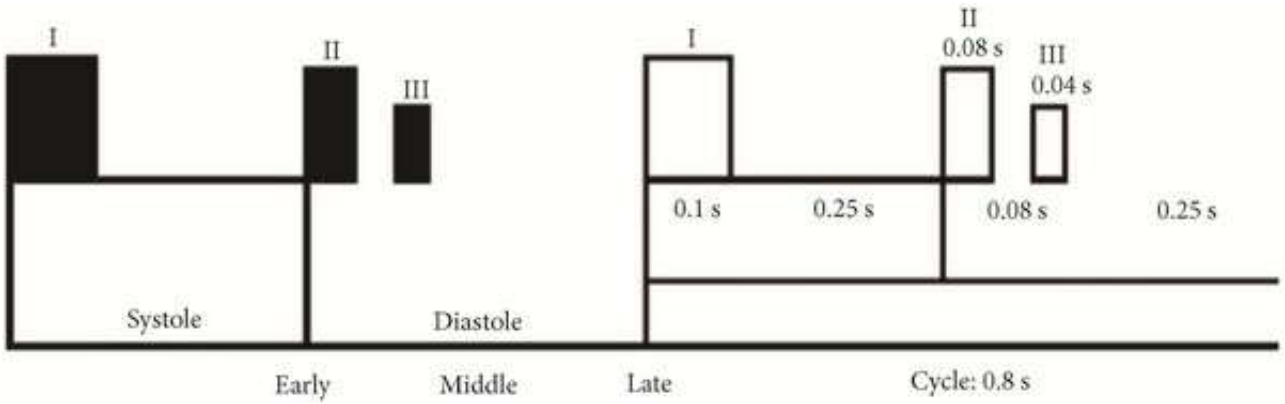
The 2nd meeting

Presenter: Xu Ruijing (Rae)

Date: 22nd,October,2025

Heart Sound and Noise

Systole收缩期	0.35 s
Diastole舒张期	0.45 s
A complete cycyle	0.8 s



- HS Datasets:
- https://www.kaggle.com/datasets/swapnilpanda/heart-sound-database
- https://data.mendeley.com/datasets/8972jxbpmp/3

Category	Noise Source
External Noise外	Speech, Motion
Internal Noise内	Respiratory, Digestive
Occasional Noise	EMI, Vocal, Physiological, Sensor, Ambient

the **murmurs** which have lower amplitude and similar characteristics to noise

[1] M. N. Ali, E.-S. A. El-Dahshan, and A. H. Yahia, "Denoising of heart sound signals using discrete wavelet transform," Circuits Syst. Signal Process., vol. 36, no. 11, pp. 4482–4497, Nov. 2017, doi: 10.1007/s00034-017-0524-7.
[2] S. Leng, R. S. Tan, K. T. C. Chai, C. Wang, D. Ghista, and L. Zhong, "The electronic stethoscope," BioMed Eng. OnLine, vol. 14, no. 1, p. 66, Jul. 2015, doi: 10.1186/s12938-015-0056-y.

Comparison

Method Category	Core Principle	Characteristics
Wavelet Transform (DWT)	Decomposes signal into different frequency bands.	Good for **non-stationary** noise. Requires manual selection of **parameters.**
Adaptive Filtering (ANC, ALE)	Real-time estimation and subtraction of noise.	Good for **stationary** noise. Need **reference noise**.
Signal Decomposition (EMD)	Self-adaptively decomposes signal into Intrinsic Mode Functions (IMFs).	Adaptive. Mode mixing and high cost.

DWT

1. Low energy and cost
2. No reference
3. Non-stationary

[2] S. Leng, R. S. Tan, K. T. C. Chai, C. Wang, D. Ghista, and L. Zhong, "The electronic stethoscope," BioMed Eng. OnLine, vol. 14, no. 1, p. 66, Jul. 2015, doi: 10.1186/s12938-015-0056-y.
[6] S. Li, F. Li, S. Tang, and W. Xiong, "A Review of Computer-Aided Heart Sound Detection Techniques," *BioMed Research International*, vol. 2020, no. 1, pp. 5846191, 2020, doi: 10.1155/2020/5846191.

Discrete Wavelet Transform (DWT)

a linear transformation method

- Multiresolution
- Windowing

1 input channel

- The DWT of a time-domain signal $s[n]$ is defined as:

$$W_x(a,b) = \sum_n \frac{1}{\sqrt{a}} s[n] \psi^* \left(\frac{n-b}{a} \right)$$

- **decomposition** of one-dimensional signals:

→ high-frequency + low-frequency

Detail Coefficient, CD 细节	Approximation Coefficient, CA 近似。宏观

[3] M. N. Ali, E.-S. A. El-Dahshan, and A. H. Yahia, "Denoising of heart sound signals using discrete wavelet transform," Circuits Syst. Signal Process., vol. 36, no. 11, pp. 4482–4497, Nov. 2017, doi: 10.1007/s00034-017-0524-7.

Mother Wavelet Function 母小波函数

Wavelet Type	Characteristics	Applications	Typical Type
Daubechies	Short support, sensitive to transients 短支撑、敏感瞬态	S1/S2 analysis, denoising	Db4， Db6
Symlets	Good symmetry, low phase distortion 对称性好、低相位畸变	Segmentation, reconstruction	Sym4， Sym6
Coiflets	Sensitive to high-order features 高阶特征敏感	Weak murmurs	Coif3， Coif5
Biorthogonal	Biorthogonal, accurate reconstruction 双正交、重构精确	Denoising, compression	Bior

需要进一步用MATLAB进行仿真和分析 → 自构小波?

[4] X. Cheng and H. Yang, "Analysis and comparison of five wavelets in heart sound signal processing," J. Nanjing Univ. Posts Telecommun. (Nat. Sci. Ed.), vol. 35, no. 1, pp. 38–46, 2015.
[5] S. Umate, "Comparison of wavelet transforms for denoising and analysis of PCG signal," unpublished manuscript, n.d.

Decomposition

■ One Level Decomposition 单层分解

original signal s[n]

↓

分析滤波 H[n] and L[n]

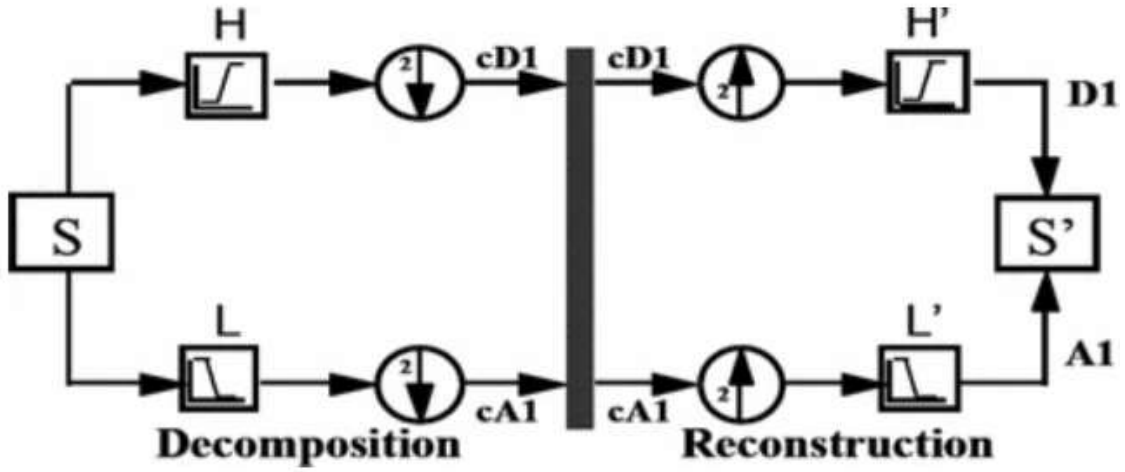
↓ **Nyquist's rule**

Downsampling by 2 二倍下采样

half of the samples are eliminated

↓

the DWT coefficients（CA，CD）



■ Inverse DWT

Upsampling by 2 二倍上采样

↓

合成滤波H′[n] and L′[n]

↓

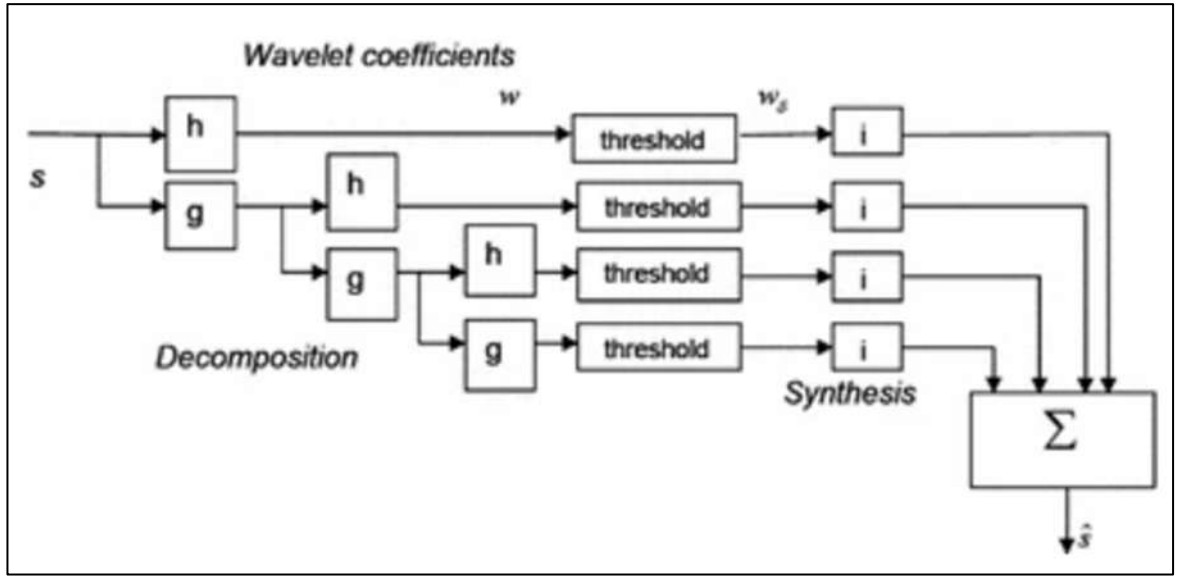
⊕ 叠加得到 重构信号S′

[3] M. N. Ali, E.-S. A. El-Dahshan, and A. H. Yahia, "Denoising of heart sound signals using discrete wavelet transform," Circuits Syst. Signal Process., vol. 36, no. 11, pp. 4482–4497, Nov. 2017, doi: 10.1007/s00034-017-0524-7.

Decomposition

■ Multi-Level Decomposition 多层分解

A_j



- For the 2nd level of decomposition:

$$A_1 = A_2 + D_2$$

- The final decomposition structure:

$$\{A_j, D_j, D_{j-1}, \cdots D_1\}$$

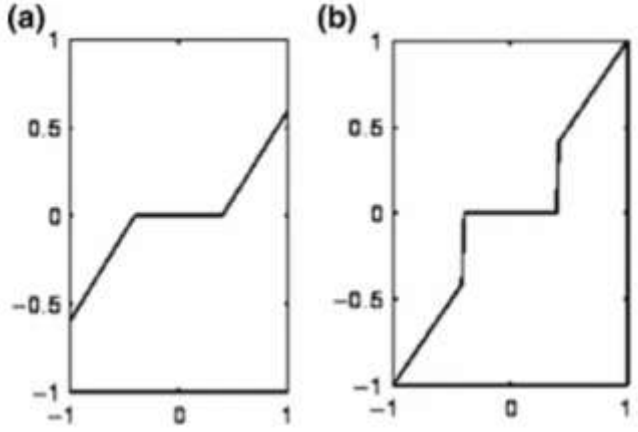
J : determines the number of frequency sub-bands频率子带数

[3] M. N. Ali, E.-S. A. El-Dahshan, and A. H. Yahia, "Denoising of heart sound signals using discrete wavelet transform," Circuits Syst. Signal Process., vol. 36, no. 11, pp. 4482–4497, Nov. 2017, doi: 10.1007/s00034-017-0524-7.

Thresholding Techniques

- Two commonest methods of Detail

Soft thresholding	Hard thresholding
Smooth transitions	Abrupt changes



✓ **Soft**

$$\mathcal{S}_T(w) = \text{sign}(w) \cdot \max(|w| - T, 0)$$

- Threshold Selection Rules

Threshold Rule	Formula / Method
VisuShrink	$\lambda = \sigma\sqrt{2\ \ln N}$
Minimax	最小化最大 均方误差MSE
Level-dependent	$\lambda_j = \sigma_j\sqrt{2\ \ln N_j}$
Bayes Shrink	$\lambda_j = \frac{\sigma^2}{\sigma_{x_j}}$
SURE Shrink	最小化 Stein 无偏风险估计(SURE) Aim：最小化估计的均方误差，无需真

[3] M. N. Ali, E.-S. A. El-Dahshan, and A. H. Yahia, "Denoising of heart sound signals using discrete wavelet transform," Circuits Syst. Signal Process., vol. 36, no. 11, pp. 4482–4497, Nov. 2017, doi: 10.1007/s00034-(
[6]D. L. Donoho, "De-noising by soft-thresholding," *IEEE Transactions on Information Theory*, vol. 41, no. 3, pp. 613–627, May 1995.

Reconstruction and Evaluation

■ Inverse DWT

各层小波系数重新合成原始信号

wavelet reconstruction filter

将各层的近似系数和细节系数逐层合成，重复 j 层得到重构信号S'

■ Performance Evaluation

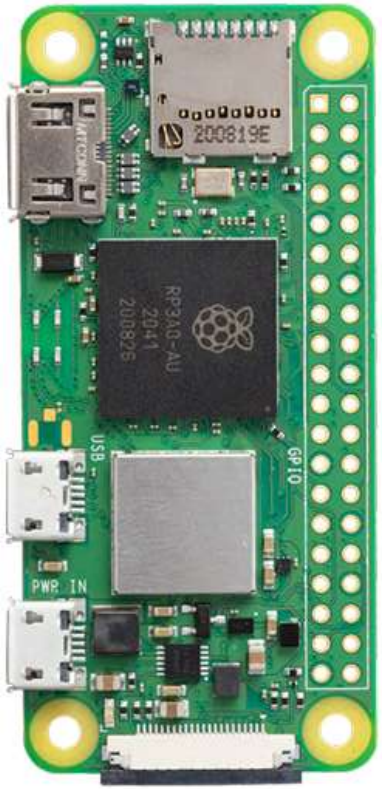
SNR	$10\log_{10}\frac{\sum_{n=0}^{N-1} s(n)^2}{(s(n)-s(\tilde{n})^2)}$
RMSE	$\sqrt{\frac{1}{N}\sum_{n=0}^{N-1}(s(n)-s(\tilde{n})^2)}$
PRD	$\sqrt{\frac{\sum_{n=0}^{N-1}(s(n)-s(\tilde{n})^2)}{\sum_{n=0}^{N-1} s(n^2)}}$

[3] M. N. Ali, E.-S. A. El-Dahshan, and A. H. Yahia, "Denoising of heart sound signals using discrete wavelet transform," Circuits Syst. Signal Process., vol. 36, no. 11, pp. 4482–4497, Nov. 2017, doi: 10.1007/s00034-017-0524-7.

In the following two weeks…

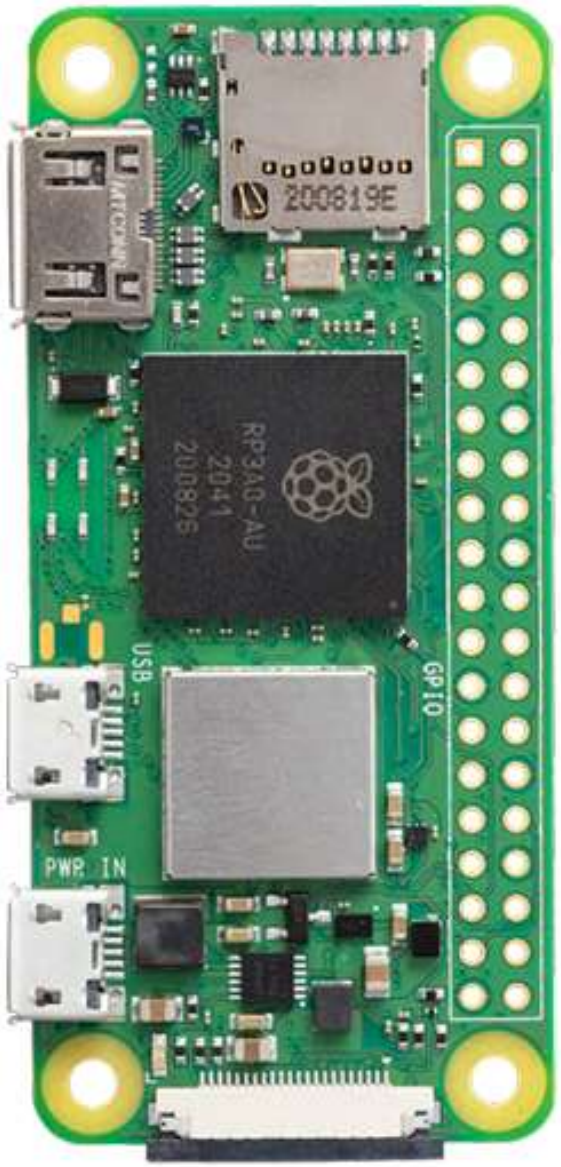
- Use MATLAB simulations to find the best mother wavelet, and then determine the optimal decomposition level.
- Review the literature related to normalization, segmentation

Fully portable continuous real-time auscultation with a soft wearable stethoscope designed for automated disease diagnosis | Science Advances

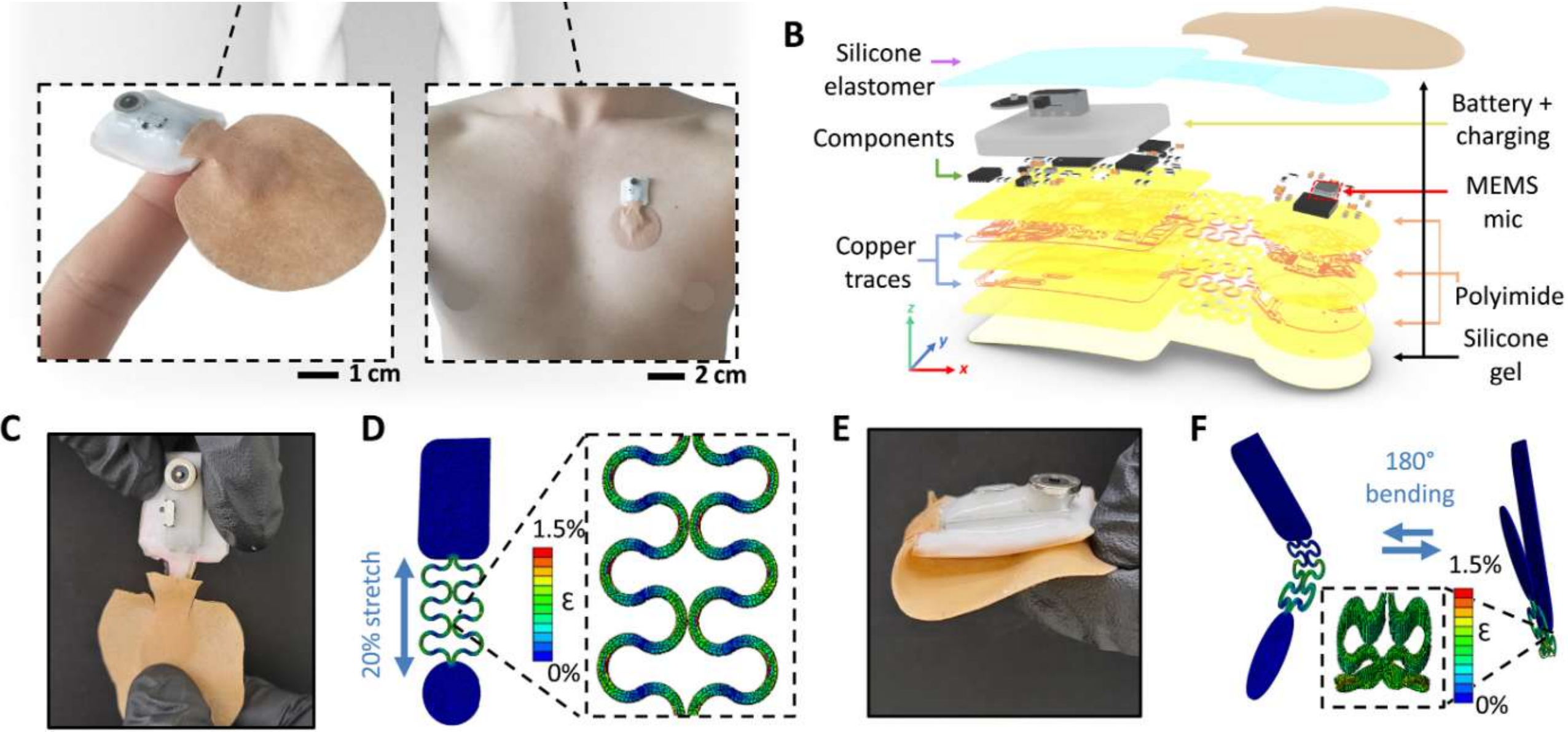


A Low-Cost AI-Empowered Stethoscope and a Lightweight Model for Detecting Cardiac and Respiratory Diseases from Lung and Heart Auscultation Sounds

github bilibili zhihu



Form factor:	65mm × 30mm
Processor:	Broadcom BCM2710A1, quad-core 64-bit SoC (Arm Cortex-A53 @ 1GHz)
Memory:	512MB LPDDR2
Connectivity:	• 2.4GHz IEEE 802.11b/g/n wireless LAN, Bluetooth 4.2, BLE, onboard antenna • 1 × USB 2.0 interface with OTG • HAT-compatible 40-pin I/O header footprint • microSD card slot • Mini HDMI port • CSI-2 camera connector
Video:	• HDMI interface • Composite video
Multimedia:	• H.264, MPEG-4 decode (1080p30) • H.264 encode (1080p30) • OpenGL ES 1.1, 2.0 graphics
Input power:	5V DC 2.5A
Operating temperature:	-20°C to +70°C

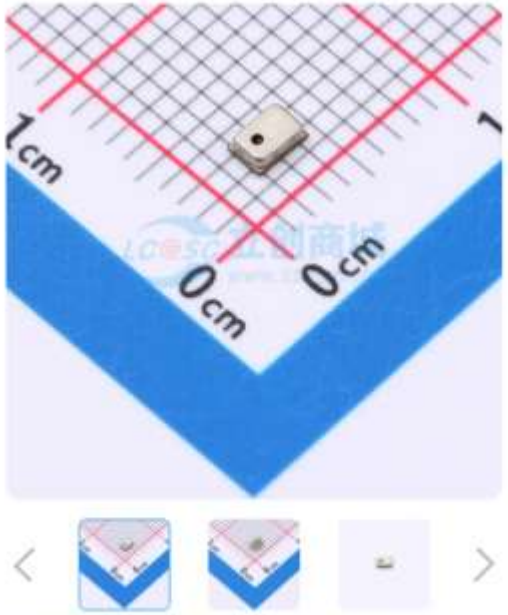

1 cm
2 cm
B
Silicone elastomer
Components
Copper traces
Battery + charging
MEMS mic
Polyimide
Silicone gel
z
y
x
C
D
20% stretch
1.5%
ε
0%
E
F
180° bending
1.5%
ε
0%

RF-star

NORDIC SEMICONDUCTOR

RF-BM-ND08

芯片型号	nRF52832
内核	64 MHz ARM® Cortex®-M4F
天线类型	PCB板载天线/ANT RF接口
RAM/Flash	64 kB/512 kB
支持协议	BLE5.0, Proprietary, ANT, NFC
工作电压	1.7 V ~ 3.6 V，推荐3.3 V
发射功率 (Max)	4 dBm
接收灵敏度	-96 dBm@1 Mbit/s BLE
功耗	TX：17.53 μA@0 dBm 1000 ms广播间隔 睡眠功耗：2.76 μA
GPIO	16
模块尺寸(mm)	11.2*15.2*1.7
工作温度	-40 °C ~ +85 °C



LMD2718T271-OA5

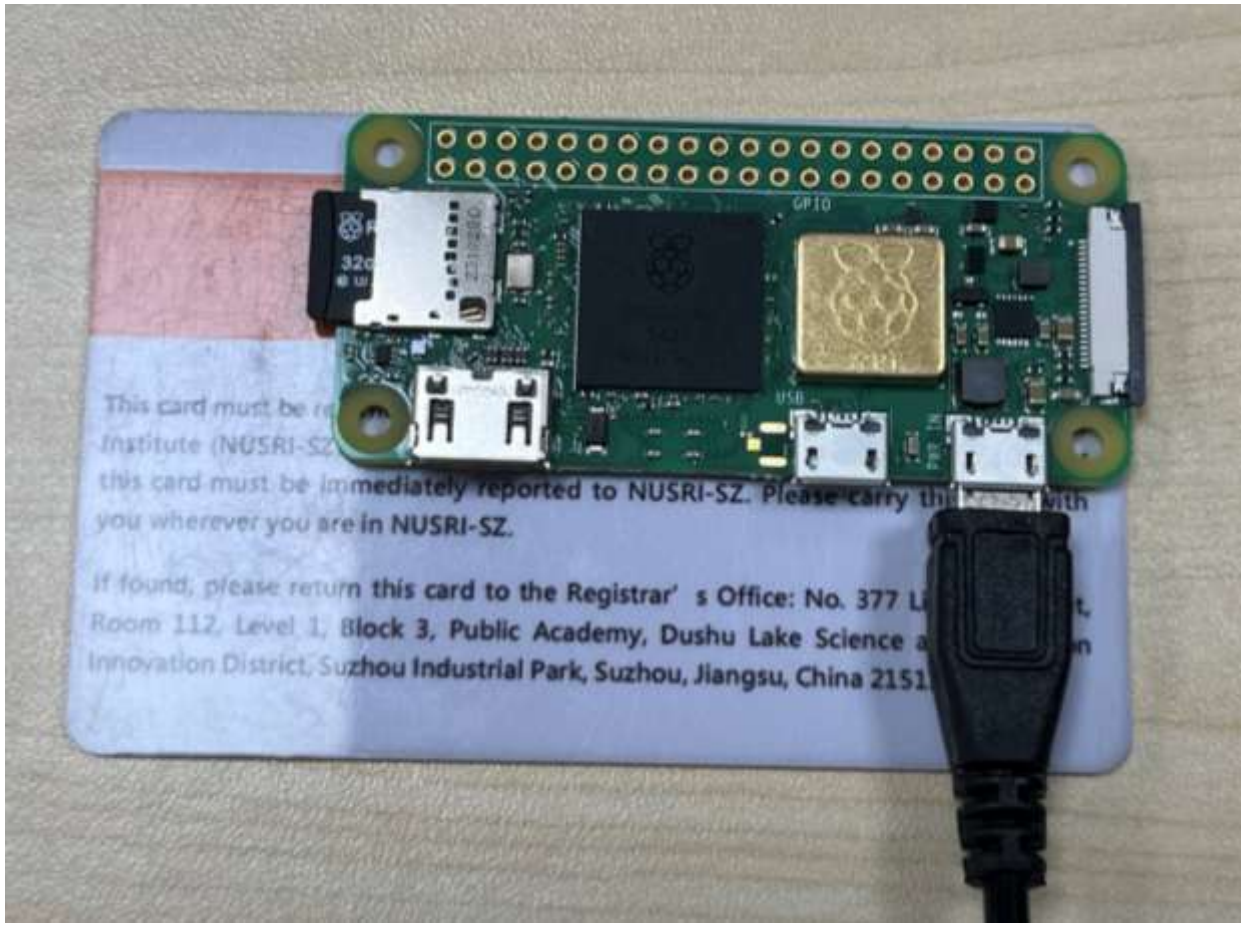
MEMSmic数字硅麦2718 上进音 s-27db±1

SMT 扩展库 | SMT补贴 嘉立创 | PCB 免费打样

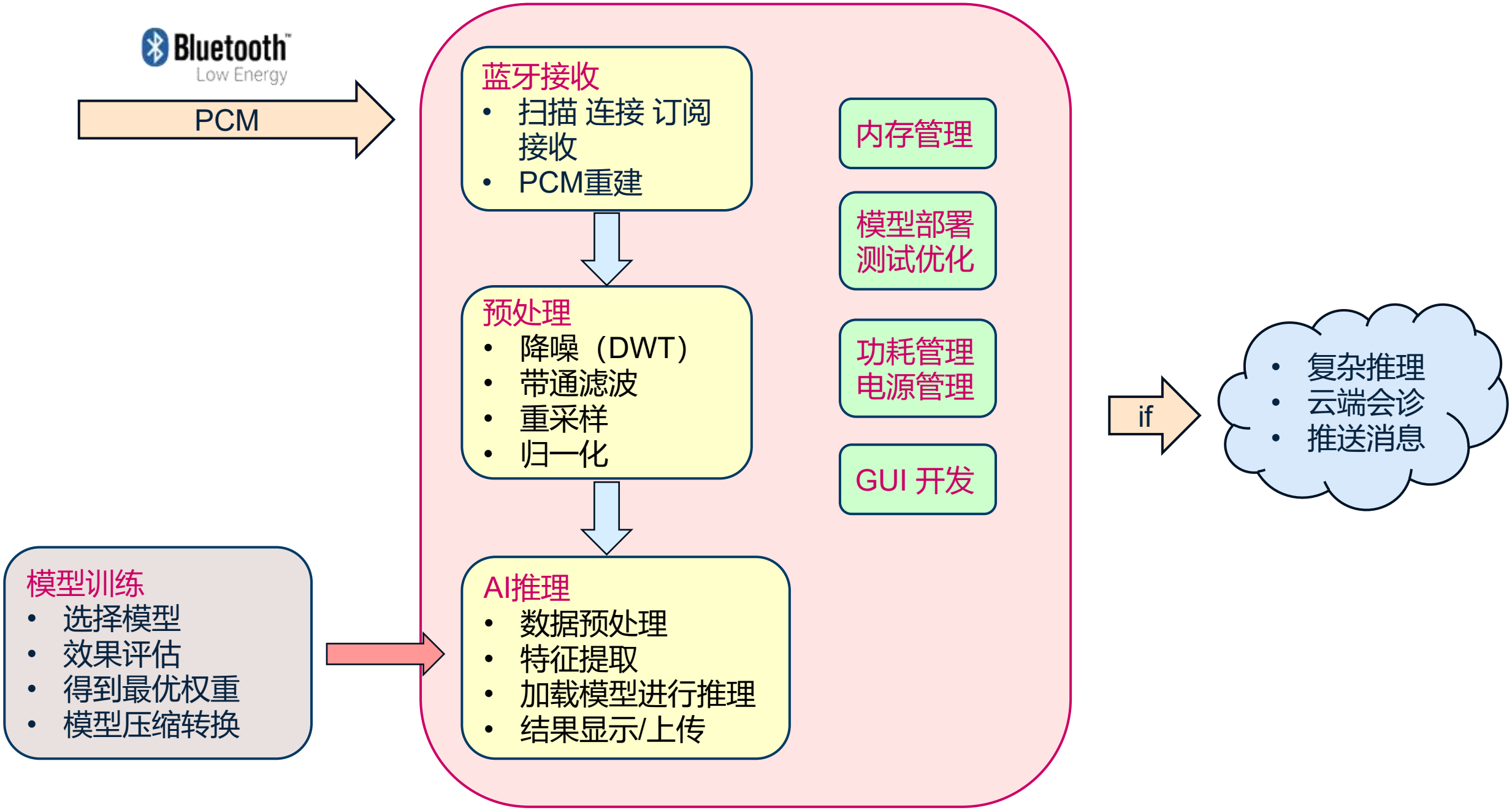
描述	2718 上进音 数字麦 -27db±1db
品牌名称	LinkMems(聆麦声学)
商品型号	LMD2718T271-OA5
商品编号	C22379480
商品封装	SMD-6P,2.8x1.9mm
包装方式	编带
商品毛重	0.104克(g)

- 灵敏度：-27dB
- SNR：54dB
- PDM输出

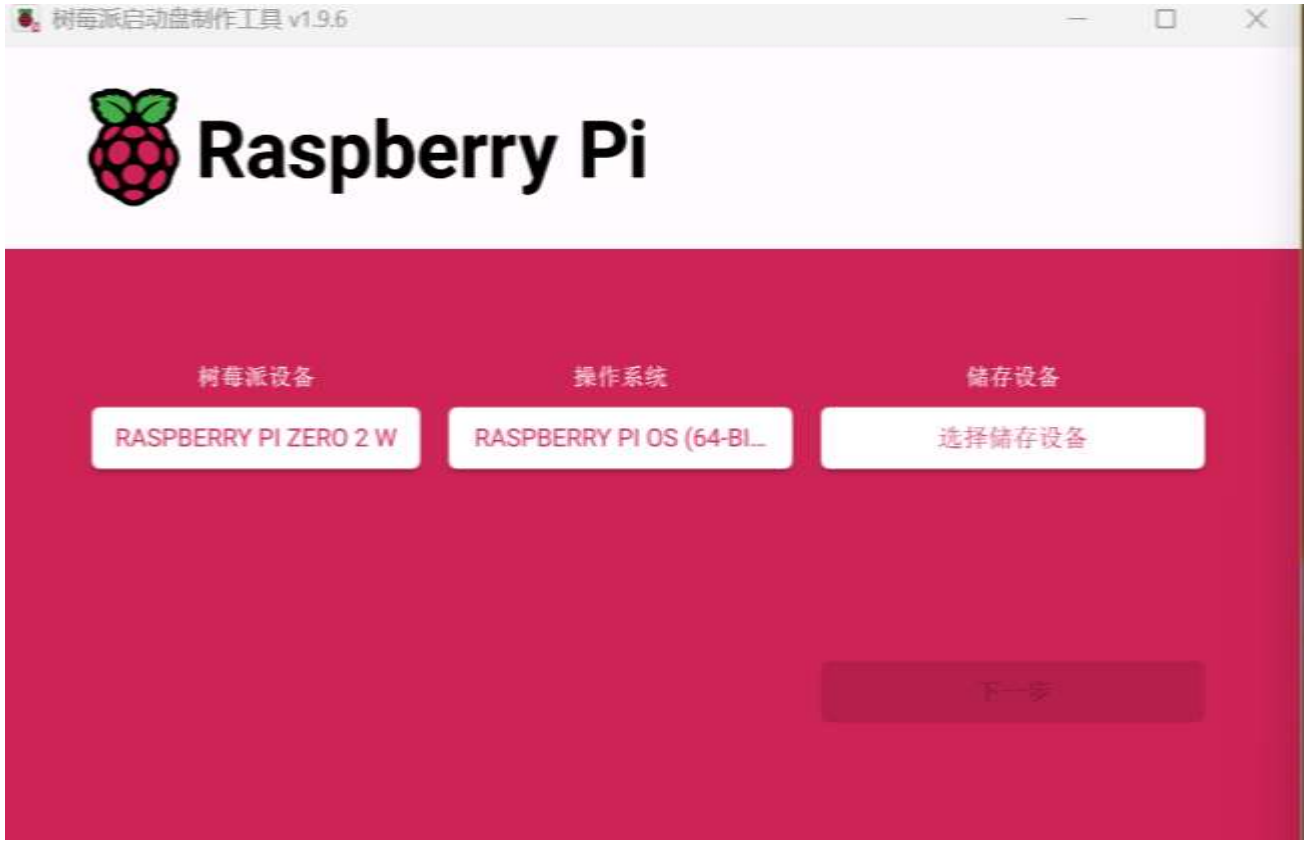
- Zero 2W（512MB）



- 4B（4GB）

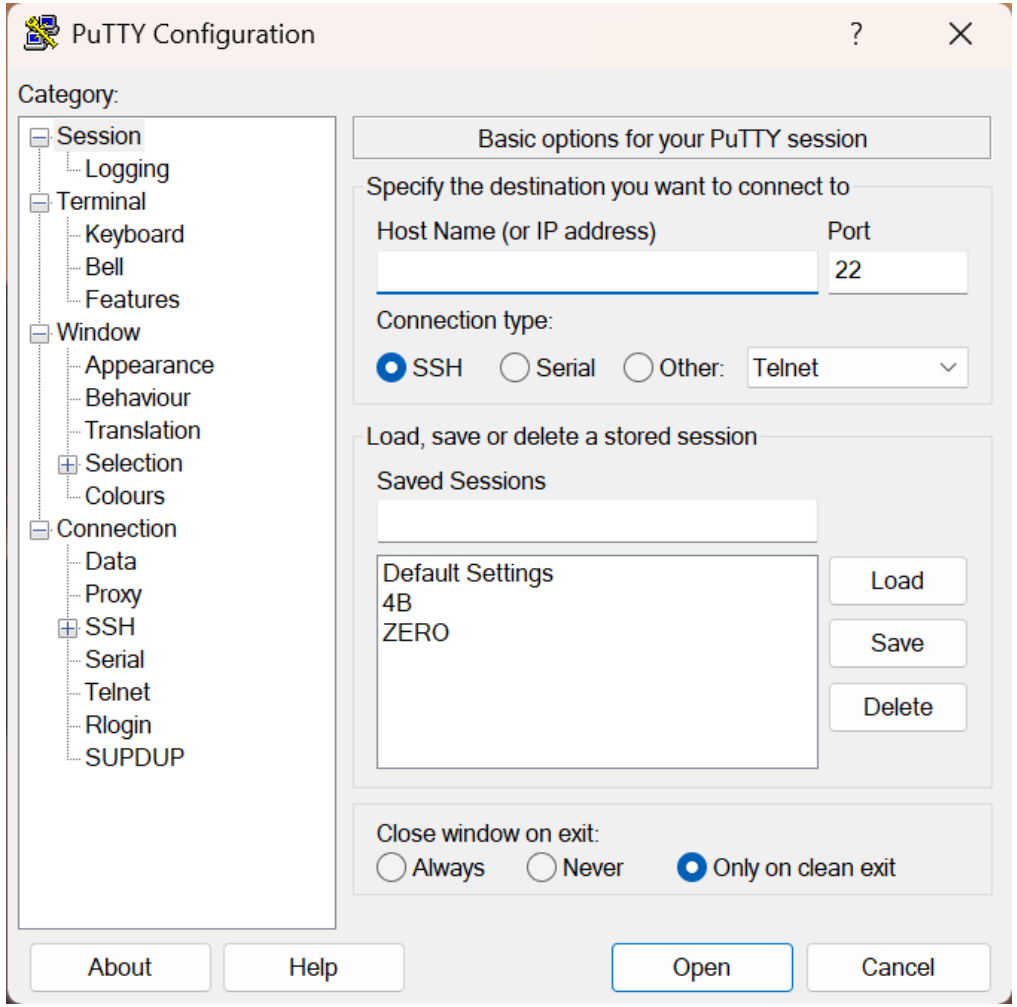

Bluetooth
Low Energy
PCM
蓝牙接收
• 扫描 连接 订阅 接收
• PCM重建
预处理
• 降噪（DWT）
• 带通滤波
• 重采样
• 归一化
AI推理
• 数据预处理
• 特征提取
• 加载模型进行推理
• 结果显示/上传
内存管理
模型部署 测试优化
功耗管理 电源管理
GUI 开发
模型训练
• 选择模型
• 效果评估
• 得到最优权重
• 模型压缩转换
if
• 复杂推理
• 云端会诊
• 推送消息

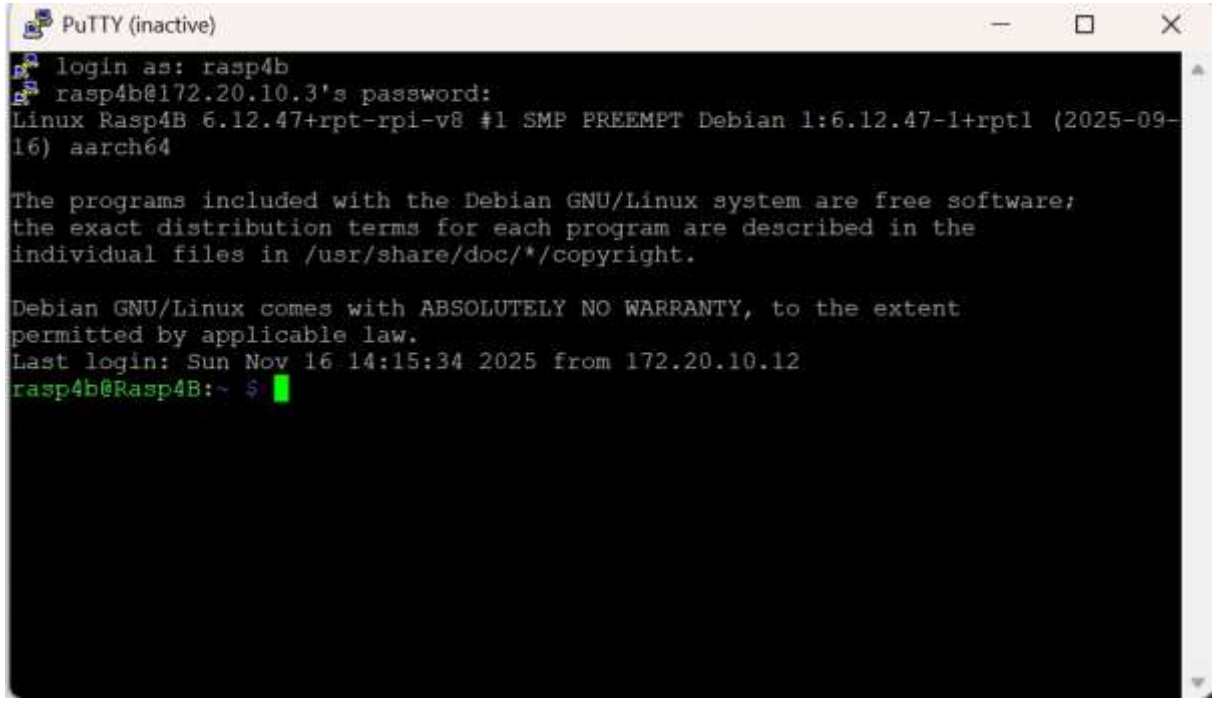
- 系统镜像烧录 ➡ microSD卡



- 手机热点

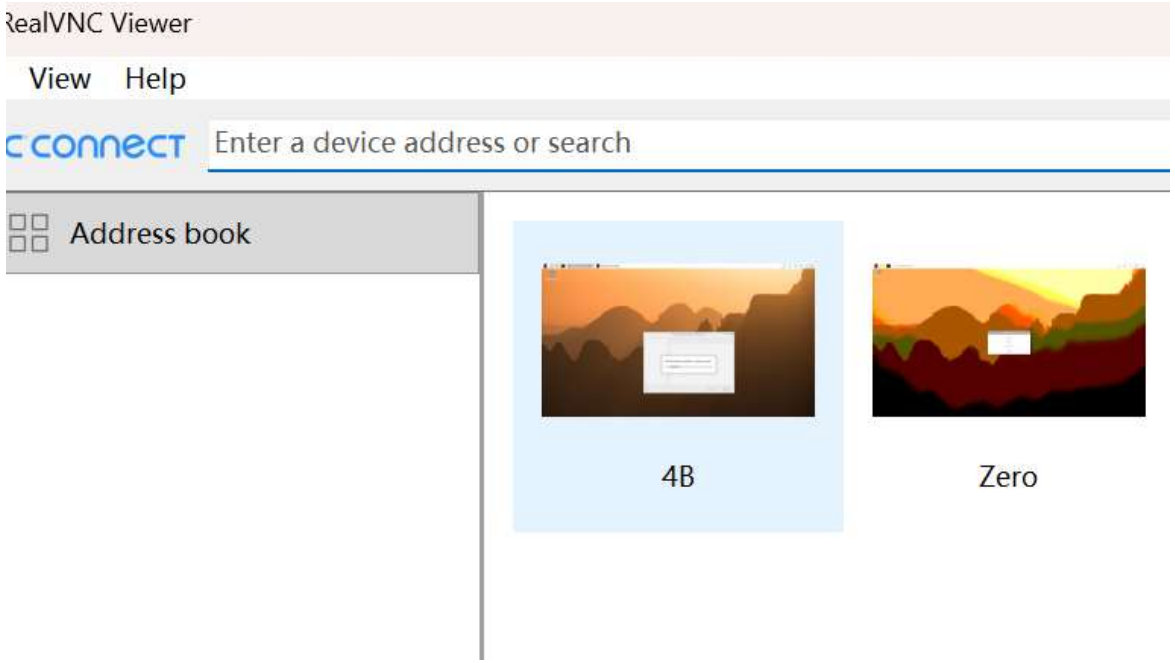
• PuTTY

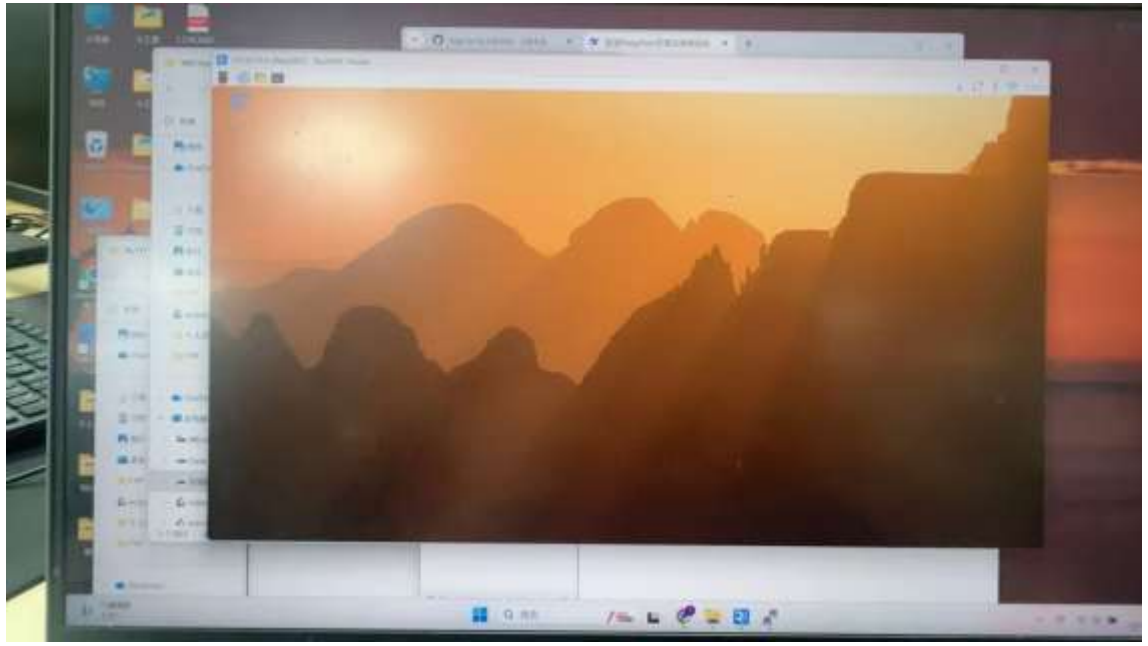




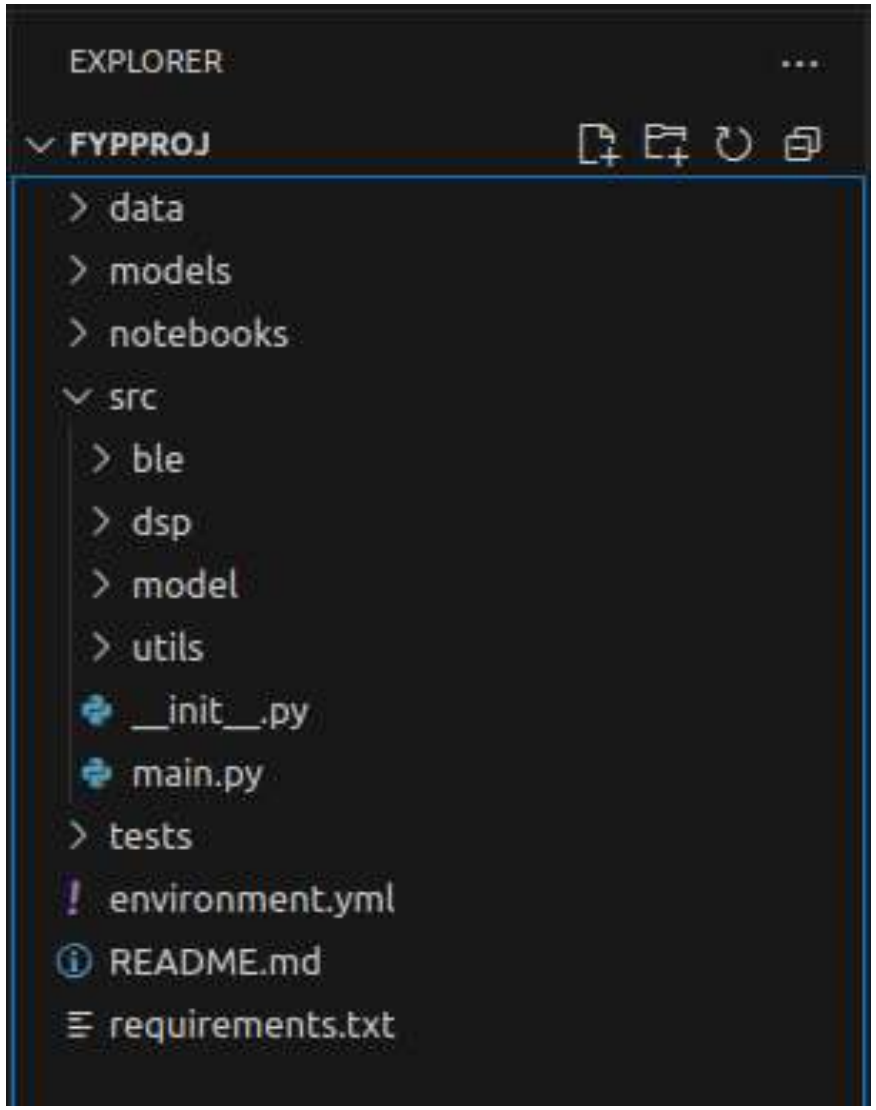
改一下VNC设置

- VNCViewer


RealVNC Viewer
View
Help
connect
Enter a device address or search
Address book
4B
Zero



- Micronmamba ← ~~Conda、系统python~~

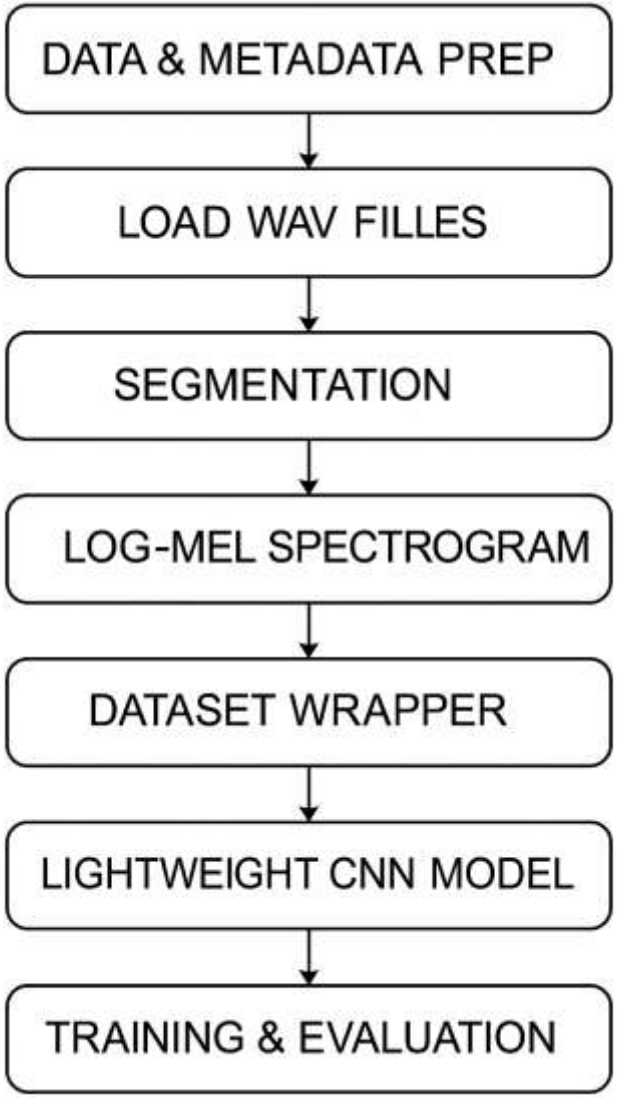


论文复现：
- 2D Bispectrum图像生成
- CNN特征提取+BDF分类

```
└── src
    ├── augment
    ├── bispectrum
    ├── ble
    ├── inference
    ├── model
    │   └── lightweight_cnn.py
    ├── preprocess
    │   ├── __pycache__
    │   │   ├── filters.cpython-310.pyc
    │   │   ├── load_wav.cpython-310.pyc
    │   │   ├── mel.cpython-310.pyc
    │   │   ├── preprocess.cpython-310.pyc
    │   │   └── segment.cpython-310.pyc
    │   ├── filters.py
    │   ├── load_metadata.py
    │   ├── load_wav.py
    │   ├── mel.py
    │   ├── segment.py
    │   └── utils
    │       ├── __pycache__
    │       ├── test_visual.py
    │       └── visualize.py
    └── train
```

https://github.com/RaeXuu/FYP

Pipeline:



```
└── src
    ├── augment
    ├── bispectrum
    ├── ble
    ├── inference
    ├── model
    │   └── lightweight_cnn.py
    ├── preprocess
    │   ├── __pycache__
    │   │   ├── filters.cpython-310.pyc
    │   │   ├── load_wav.cpython-310.pyc
    │   │   ├── mel.cpython-310.pyc
    │   │   ├── preprocess.cpython-310.pyc
    │   │   └── segment.cpython-310.pyc
    │   ├── filters.py
    │   ├── load_metadata.py
    │   ├── load_wav.py
    │   ├── mel.py
    │   ├── segment.py
    │   └── utils
    │       ├── __pycache__
    │       ├── test_visual.py
    │       └── visualize.py
    └── train
```

https://github.com/RaeXuu/FYP

1. Metadata prepare

DATA & METADATA PREP

(src/preprocess/load_metadata.py)

- 从 DataSet1(set_a.csv)、DataSet2(set_b.csv) 读取信息
- 合并成 metadata1.csv (路径 + 标签 + 来源)

```
rae@Rae:/mnt/d/FypProj$ /home/rae/micromamba/envs/FypProj/bin/python /mnt/d/FypProj/src/preprocess/load_me
  dataset                      fname  ...                                          filepath duration
0       a  artifact__201012172012.wav  ...  /mnt/d/FypProj/data/raw/DataSet1/set_a/artifac...      9.0
1       a  artifact__201105040918.wav  ...  /mnt/d/FypProj/data/raw/DataSet1/set_a/artifac...      9.0
2       a  artifact__201105041959.wav  ...  /mnt/d/FypProj/data/raw/DataSet1/set_a/artifac...      9.0
3       a  artifact__201105051017.wav  ...  /mnt/d/FypProj/data/raw/DataSet1/set_a/artifac...      9.0
4       a  artifact__201105060108.wav  ...  /mnt/d/FypProj/data/raw/DataSet1/set_a/artifac...      9.0

[5 rows x 8 columns]
总样本数: 832
已保存 metadata 到: /mnt/d/FypProj/data/metadata1.csv
```

保存成.csv格式

```
└── src
    ├── augment
    ├── bispectrum
    ├── ble
    ├── inference
    ├── model
    │   └── lightweight_cnn.py
    ├── preprocess
    │   ├── __pycache__
    │   │   ├── filters.cpython-310.pyc
    │   │   ├── load_wav.cpython-310.pyc
    │   │   ├── mel.cpython-310.pyc
    │   │   ├── preprocess.cpython-310.pyc
    │   │   └── segment.cpython-310.pyc
    │   ├── filters.py
    │   ├── load_metadata.py
    │   ├── load_wav.py
    │   ├── mel.py
    │   ├── segment.py
    │   └── utils
    │       ├── __pycache__
    │       ├── test_visual.py
    │       └── visualize.py
    └── train
```

https://github.com/RaeXuu/FYP

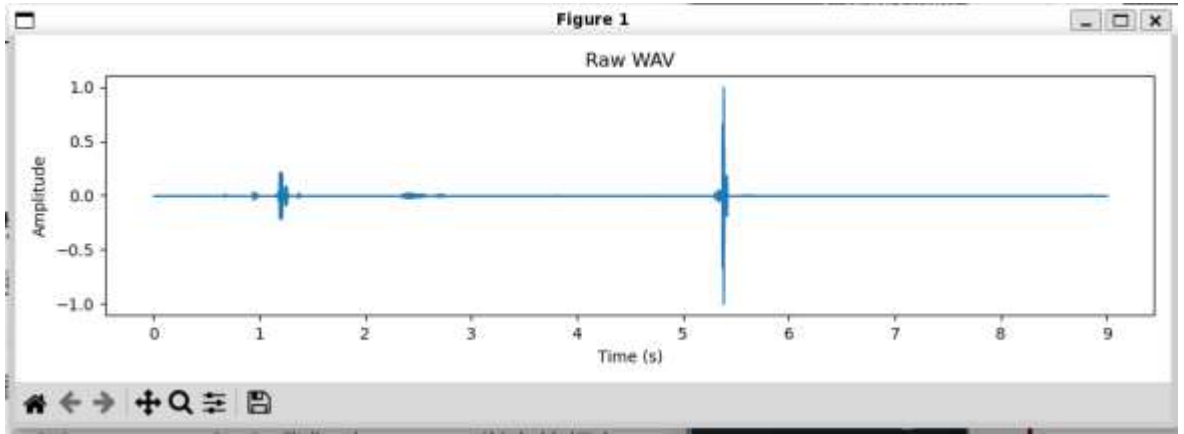
2. Load wav files

LOAD WAV FILES

(src/preprocess/load_wav.py)

- 使用 librosa 读取 .wav
- 统一采样率 4000 Hz
- 归一化到 [-1, 1]

```
rae@Rae:/mnt/d/FypProj$ /hc
测试加载成功，返回数量: 3
第一个样本信息:
  采样率: 4000
  音频长度: 36000
```



```
└── src
    ├── augment
    ├── bispectrum
    ├── ble
    ├── inference
    ├── model
    │   └── lightweight_cnn.py
    ├── preprocess
    │   ├── __pycache__
    │   │   ├── filters.cpython-310.pyc
    │   │   ├── load_wav.cpython-310.pyc
    │   │   ├── mel.cpython-310.pyc
    │   │   ├── preprocess.cpython-310.pyc
    │   │   └── segment.cpython-310.pyc
    │   ├── filters.py
    │   ├── load_metadata.py
    │   ├── load_wav.py
    │   ├── mel.py
    │   ├── segment.py
    │   └── utils
    │       ├── __pycache__
    │       ├── test_visual.py
    │       └── visualize.py
    └── train
```

https://github.com/RaeXuu/FYP

3. Band-Pass filters

BAND-PASS FILTER

(src/preprocess/filters.py)

- Butterworth 4阶滤波器
- 频率范围 20 – 400 Hz
- 去除低频漂移与高频噪声

```
rae@Rae:/mnt/d/FypProj$ /home/rae/micro
测试样本路径: /mnt/d/FypProj/data/raw/D
原始信号长度: 36000
滤波后信号长度: 36000
原始能量: 0.00039297497
滤波后能量: 0.00028672320892899884
带通滤波测试完成 ✅
```

```
└── src
    ├── augment
    ├── bispectrum
    ├── ble
    ├── inference
    ├── model
    │   └── lightweight_cnn.py
    ├── preprocess
    │   ├── __pycache__
    │   │   ├── filters.cpython-310.pyc
    │   │   ├── load_wav.cpython-310.pyc
    │   │   ├── mel.cpython-310.pyc
    │   │   ├── preprocess.cpython-310.pyc
    │   │   └── segment.cpython-310.pyc
    │   ├── filters.py
    │   ├── load_metadata.py
    │   ├── load_wav.py
    │   ├── mel.py
    │   ├── segment.py
    │   └── utils
    │       ├── __pycache__
    │       ├── test_visual.py
    │       └── visualize.py
    └── train
```

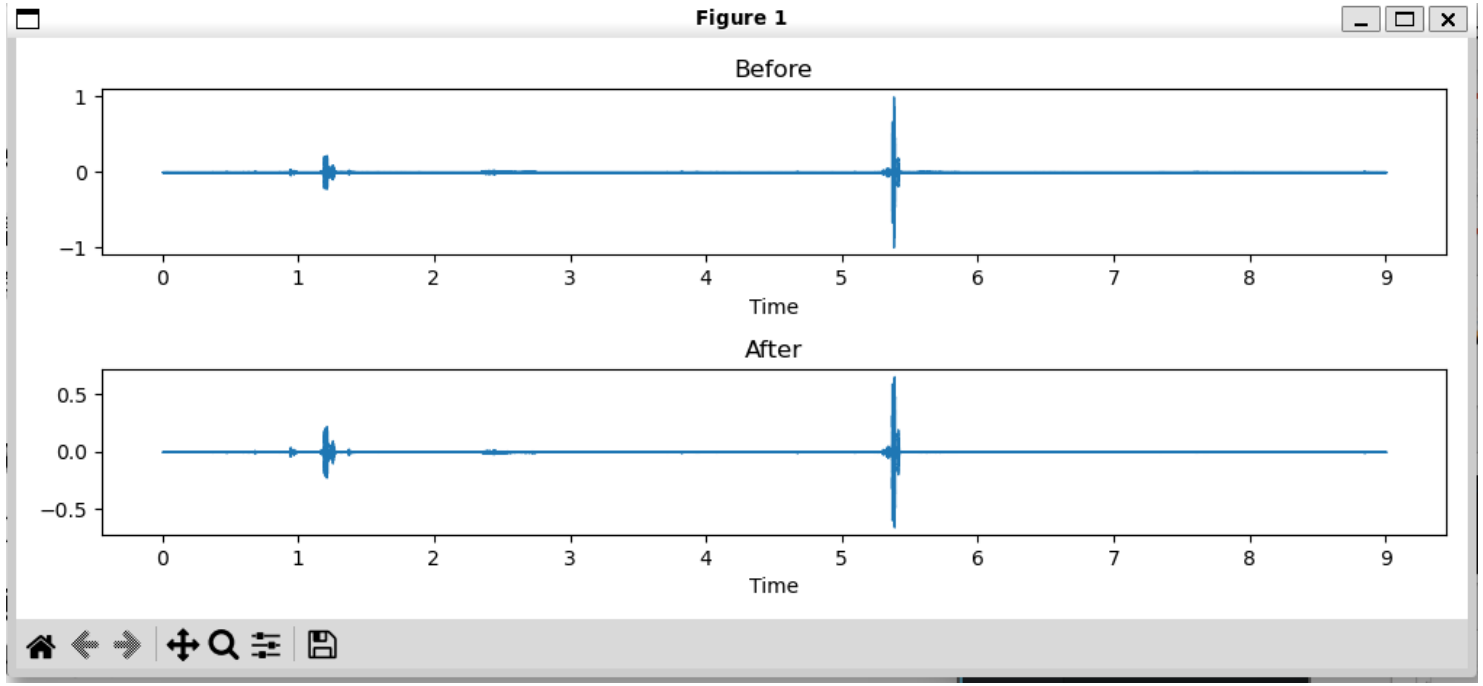
https://github.com/RaeXuu/FYP

3. Band-Pass filters

BAND-PASS FILTER

(src/preprocess/filters.py)

- Butterworth 4阶滤波器
- 频率范围 20 – 400 Hz
- 去除低频漂移与高频噪声



```
└── src
    ├── augment
    ├── bispectrum
    ├── ble
    ├── inference
    ├── model
    │   └── lightweight_cnn.py
    ├── preprocess
    │   ├── __pycache__
    │   │   ├── filters.cpython-310.pyc
    │   │   ├── load_wav.cpython-310.pyc
    │   │   ├── mel.cpython-310.pyc
    │   │   ├── preprocess.cpython-310.pyc
    │   │   └── segment.cpython-310.pyc
    │   ├── filters.py
    │   ├── load_metadata.py
    │   ├── load_wav.py
    │   ├── mel.py
    │   ├── segment.py
    │   └── utils
    │       ├── __pycache__
    │       ├── test_visual.py
    │       └── visualize.py
    └── train
```

https://github.com/RaeXuu/FYP

4. Segmentations

SEGMENTATION

(src/preprocess/segment.py)

- 将每条心音切成 2 秒片段（8 000 sample）
- 不足补零，超出分段
- 每段独立带标签存储

```
rae@Rae:/mnt/d/FypProj$
测试样本: /mnt/d/FypPro
切片数量: 5
单个片段长度: 8000
理论应为: 8000
切片测试完成 ✅
```

```
└── src
    ├── augment
    ├── bispectrum
    ├── ble
    ├── inference
    ├── model
    │   └── lightweight_cnn.py
    ├── preprocess
    │   ├── __pycache__
    │   │   ├── filters.cpython-310.pyc
    │   │   ├── load_wav.cpython-310.pyc
    │   │   ├── mel.cpython-310.pyc
    │   │   ├── preprocess.cpython-310.pyc
    │   │   └── segment.cpython-310.pyc
    │   ├── filters.py
    │   ├── load_metadata.py
    │   ├── load_wav.py
    │   ├── mel.py
    │   ├── segment.py
    │   └── utils
    │       ├── __pycache__
    │       ├── test_visual.py
    │       └── visualize.py
    └── train
```

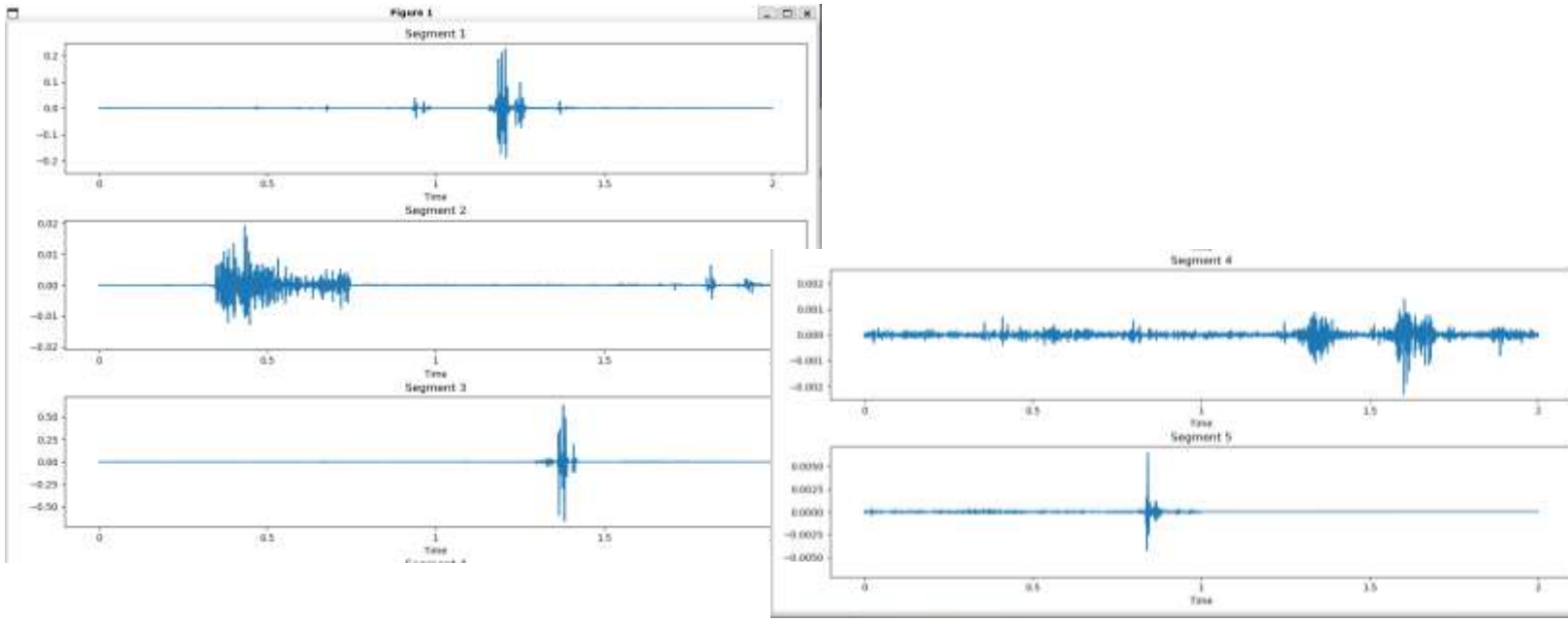
https://github.com/RaeXuu/FYP

4. Segmentations

SEGMENTATION

(src/preprocess/segment.py)

- 将每条心音切成 2 秒片段（8 000 sample）
- 不足补零，超出分段
- 每段独立带标签存储



```
└── src
    ├── augment
    ├── bispectrum
    ├── ble
    ├── inference
    ├── model
    │   └── lightweight_cnn.py
    ├── preprocess
    │   ├── __pycache__
    │   │   ├── filters.cpython-310.pyc
    │   │   ├── load_wav.cpython-310.pyc
    │   │   ├── mel.cpython-310.pyc
    │   │   ├── preprocess.cpython-310.pyc
    │   │   └── segment.cpython-310.pyc
    │   ├── filters.py
    │   ├── load_metadata.py
    │   ├── load_wav.py
    │   ├── mel.py
    │   ├── segment.py
    │   └── utils
    │       ├── __pycache__
    │       ├── test_visual.py
    │       └── visualize.py
    └── train
```

https://github.com/RaeXuu/FYP

5. Mel spectrogram

LOG-MEL SPECTROGRAM

(src/preprocess/mel.py)

- 生成 64x64 Mel 特征图
- 对数能量刻度 (log-mel)
- 输出 2D 图像张量供 CNN 使用

```
rae@Rae:/mnt/d/FypProj$ /h
测试样本: /mnt/d/FypProj/d
单段长度: 8000
Log-Mel shape: (64, 64)
Mel 测试完成 ✅
```

```
└── src
    ├── augment
    ├── bispectrum
    ├── ble
    ├── inference
    ├── model
    │   └── lightweight_cnn.py
    ├── preprocess
    │   ├── __pycache__
    │   │   ├── filters.cpython-310.pyc
    │   │   ├── load_wav.cpython-310.pyc
    │   │   ├── mel.cpython-310.pyc
    │   │   ├── preprocess.cpython-310.pyc
    │   │   └── segment.cpython-310.pyc
    │   ├── filters.py
    │   ├── load_metadata.py
    │   ├── load_wav.py
    │   ├── mel.py
    │   ├── segment.py
    │   └── utils
    │       ├── __pycache__
    │       ├── test_visual.py
    │       └── visualize.py
    └── train
```

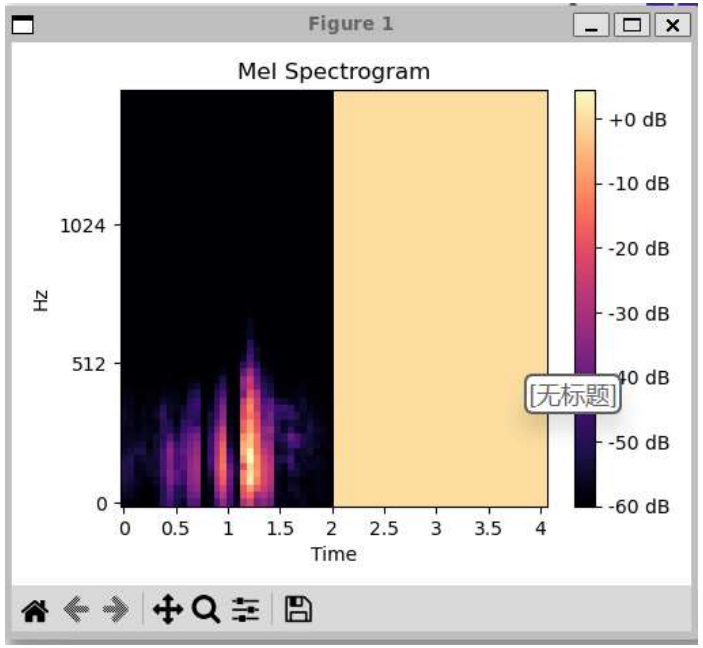
https://github.com/RaeXuu/FYP

5. Mel spectrogram

LOG-MEL SPECTROGRAM

(src/preprocess/mel.py)

- 生成 64x64 Mel 特征图
- 对数能量刻度 (log-mel)
- 输出 2D 图像张量供 CNN 使用



```
└── src
    ├── augment
    ├── bispectrum
    ├── ble
    ├── inference
    ├── model
    │   └── lightweight_cnn.py
    ├── preprocess
    │   ├── __pycache__
    │   │   ├── filters.cpython-310.pyc
    │   │   ├── load_wav.cpython-310.pyc
    │   │   ├── mel.cpython-310.pyc
    │   │   ├── preprocess.cpython-310.pyc
    │   │   └── segment.cpython-310.pyc
    │   ├── filters.py
    │   ├── load_metadata.py
    │   ├── load_wav.py
    │   ├── mel.py
    │   ├── segment.py
    │   └── utils
    │       ├── __pycache__
    │       ├── test_visual.py
    │       └── visualize.py
    └── train
```

https://github.com/RaeXuu/FYP

6. Dataset wrapper

DATASET WRAPPER

(src/train/dataset.py)

- 统一加载 metadata → WAV → Filter → Segment → Mel
- 返回 (Tensor[1, 64, 64], label_index)
- 便于 DataLoader 直接使用

```
rae@Rae:/mnt/d/FypProj$ /home/rae/micromam
[Dataset] 总切片数: 3247
Dataset 总长度（切片数）: 3247
单个 Mel shape: torch.Size([1, 64, 64])
标签 index: 2
Dataset 自测完成 ✅
```

```
└── src
    ├── augment
    ├── bispectrum
    ├── ble
    ├── inference
    ├── model
    |   └── lightweight_cnn.py
    ├── preprocess
    |   ├── __pycache__
    |   |   ├── filters.cpython-310.pyc
    |   |   ├── load_wav.cpython-310.pyc
    |   |   ├── mel.cpython-310.pyc
    |   |   ├── preprocess.cpython-310.pyc
    |   |   └── segment.cpython-310.pyc
    |   ├── filters.py
    |   ├── load_metadata.py
    |   ├── load_wav.py
    |   ├── mel.py
    |   ├── segment.py
    |   └── utils
    |       ├── __pycache__
    |       ├── test_visual.py
    |       └── visualize.py
    └── train
```

https://github.com/RaeXuu/FYP

7. Lightweight CNN model

LIGHTWEIGHT CNN MODEL

(src/model/lightweight_cnn.py)

- Conv + 3 x Depthwise Separable Conv
- Global Avg Pool → FC → Softmax
- 参数量 ≈ 13 k，适合嵌入式部署

“1 个普通卷积 + 3 个深度可分离卷积 + 全局池化 + 全连接”

```
└── src
    ├── augment
    ├── bispectrum
    ├── ble
    ├── inference
    ├── model
    │   └── lightweight_cnn.py
    ├── preprocess
    │   ├── __pycache__
    │   │   ├── filters.cpython-310.pyc
    │   │   ├── load_wav.cpython-310.pyc
    │   │   ├── mel.cpython-310.pyc
    │   │   ├── preprocess.cpython-310.pyc
    │   │   └── segment.cpython-310.pyc
    │   ├── filters.py
    │   ├── load_metadata.py
    │   ├── load_wav.py
    │   ├── mel.py
    │   ├── segment.py
    │   └── utils
    │       ├── __pycache__
    │       ├── test_visual.py
    │       └── visualize.py
    └── train
```

https://github.com/RaeXuu/FYP

7. Lightweight CNN model

LIGHTWEIGHT CNN MODEL

(src/model/lightweight_cnn.py)

- Conv + 3 x Depthwise Separable Conv
- Global Avg Pool → FC → Softmax
- 参数量 ≈ 13 k，适合嵌入式部署

```
rae@Rae:/mnt/d/FypProj$
/home/rae/micromamba/envs/FypProj/bin/python
/mnt/d/FypProj/src/model/lightweight_cnn.py
输入形状: torch.Size([2, 1, 64, 64])
输出形状: torch.Size([2, 4])
参数量: 13124
LightweightCNN 自测完成 ✅
```

Future Work：

- Focus on the development of the bluetooth chestpiece first.
- Bluetooth Function Debugging
- Model training, Evaluation & Testing

开始的地方：

```
==========================
Final Test Loss: 0.7897
Final Test Acc : 0.7302
==========================
```

先让模型的准确率可观并且可靠。然后再考虑可部署性，进行模型压缩（Model Compression）

Feature：

Results：

特征	Best Val Acc	收敛情况	稳定性
Mel	**0.8370**	快	中后期略有波动
Wavelet (CWT)	**0.8238**	稍慢	很稳
Bispectrum / Bicoherence	**0.6432**	慢	不稳定

Conclusions：

特征	本质是什么	捕捉的信息
Mel	类语音的时频表示（STFT + Mel 滤波）	频带能量 + 时间结构
Wavelet (CWT)	多尺度时频分析	非平稳、瞬态结构
Bispectrum	高阶谱（非线性统计）	相位耦合、非高斯性

Model：

Lightweight CNN	Transformer
========== Final Test Loss: 0.7897 Final Test Acc : 0.7302 ==========	========== Final Test Loss: 0.9019 Final Test Acc : 0.6862 ==========

Augment 数据增强

```python
def waveform_augment(
    x,
    prob_gain=0.5,
    prob_noise=0.5,
    prob_shift=0.5,
):
    """
    组合增广（按概率触发）
    """
    if np.random.rand() < prob_gain:
        x = random_gain(x)

    if np.random.rand() < prob_noise:
        x = add_gaussian_noise(x)

    if np.random.rand() < prob_shift:
        x = random_time_shift(x)

    return x
```

随机幅值缩放
（模拟按压力读/个体差异）

加少量高斯噪声

随机时间平移

Result of Augment：

设置	Best Val Acc
Mel + augment ON	0.8370
Mel + augment OFF	0.8899 ↑

猜测：心音的相位结构不适合做数据增强。Time shift 和 noise可能会破坏有效的数据

One Epoch Evalute：

```
Epoch [25/25]
Train Loss: 0.1460 | Train Acc: 0.9499

Val   Loss: 0.9729 | Val   Acc: 0.7554
```

```
[Validation] Confusion Matrix:
[[ 36   3   0   0   1]
 [  9  18   0   0  22]
 [  0   0   2   1  35]
 [  0   0   1 118  47]
 [  7   3   1  39 348]]
```

```
[Validation] Classification Report:
              precision    recall  f1-score   support

    artifact     0.6923    0.9000    0.7826        40
    extrahls     0.7500    0.3673    0.4932        49
  extrastole     0.5000    0.0526    0.0952        38
      murmur     0.7468    0.7108    0.7284       166
      normal     0.7682    0.8744    0.8179       398

    accuracy                         0.7554       691
   macro avg     0.6915    0.5810    0.5835       691
weighted avg     0.7426    0.7554    0.7316       691
```

```
✅ New best model saved! Acc=0.7554
```

config.yaml → ablation

config.yaml

```yaml
data:
  sample_rate: 2000  # 4000
  segment_length: 2.0  # 2.5 3.0
  overlap: 0.5  # 0.0
  bandpass:
    low: 20
    high: 400

mel:
  n_fft: 256  # 512
  win_length: 256  # 512
  hop_length: 96  # 128 # 64 # 80
  n_mels: 32  # 40
  fmin: 20
  fmax: 400
  power: 2.0
```

Test Set

一上test set，发现准确率和之前比直接崩了。

发现问题：一条音频的几个切片进了不同的train/validation set。造成了提前泄露

解决办法：

```
# ====================
# Group split by fname
# ====================
```

目前情况：

```
============================================================
[TEST] Loading best model and evaluating on test set...
============================================================

Test  Loss: 0.8751 | Test  Acc: 0.7898

[Test] Confusion Matrix:
[[ 16   0   0   0   0]
 [  1  17   0   0   5]
 [  0   0   0   0  24]
 [  0   0   0  62  27]
 [  0   0   2  15 183]]
```

Inference

Dummy input 纯噪音：

```
(FypProj) rae@Rae:/mnt/d/FypProj$ python -m src.inference.run_inference
Predicted class: 0
Probabilities: [9.9980062e-01 1.9940108e-04 7.4892332e-12 9.6137933e-09 1.0587990e-08]
```

还没有用真实的音频推理过

Deployment Benchmark 可部署性考量

类别	指标	是否可量化
Model Size	参数量 (#Params)	可以
	权重文件大小 (MB)	可以
Latency	推理延迟 (ms)	可以
Complexity	MACs / FLOPs	可以（估算）
Energy Consumption	每次推理时间	可以（间接）
Accuracy	FP32 / INT8	可以（后续）
Runtime Memory	Activations 内存	近似

Model Compression 模型压缩

- Quantization
- Pruning
- Distillation

Benchmark 脚本

```
(FypProj) rae@Rae:/mnt/d/FypProj$ python -m src.inference.benchmark_model
==== Deployment Benchmark ====
[Params] 13,253
[File ] 0.07 MB
[MACs ] 35.19 M
[CPU  ] Latency = 0.962 ms / sample
[TPS  ] 1039.02 samples/s
==============================
```

Future Work：

- Deployment on Raspberry Pi
- Connect with Chestpiece
- Debug

改动一：Data Set 更换

- 一开始选择了两个数据集，但只用了其中一个
  - （开始时使用的数据集）PASCAL Classifying Heart Sounds Challenge 2011 (CHSC2011)

    Classes: five

    Time length: 1 ~ 9 seconds

    Sampling rate: mostly 4k Hz

  - The PhysioNet/Computing in Cardiology Challenge 2016

    3,153 samples

    Classes: two (normal, abnormal)

    Time length: 5 ~ 120 seconds

    Sampling rate: 2k Hz

改动二：模型更复杂 加入了Attention 训练策略改变

方案	核心维度	对心音的“杀伤力”	部署友好度	毕设亮点
SE Att	仅通道 (Frequency)	能过滤噪声频段，但对“漏拍”或“节奏紊乱”不敏感。	⭐⭐⭐⭐⭐	基础款
CBAM	通道 + 空间 (Mixed)	综合性能强，但空间注意力在 32×64 这种小图上容易产生冗余计算。	⭐⭐⭐	经典款
Coord Att	通道 + 时间 + 频率	极强。 分别对频率轴和时间轴建模，完美匹配心音的周期性。	⭐⭐⭐⭐⭐	进阶款（推荐）
Temporal Att	仅时间 (Time)	擅长捕捉节奏，但容易忽视不同频段杂音的区别。	⭐⭐⭐⭐	垂直款

改动二：模型更复杂 加入了Attention 训练策略改变

- CNN的通道数加大
- 训练策略：加入了数据集增强、优化器加入权重衰减、加入学习率调度器

特性	更改前	更改后
模型版本	标准版 LightweightCNN	CoordAtt 版
输入形状	[2, 1, 64, 64]	[2, 1, 32, 64]
输出形状	[2, 4]	[2, 2]
参数量	13.12 k	65.12 k
验证集准确率	0.8797	0.8924
验证集损失	0.3437	0.2624
收敛轮次	第16轮	第6轮

改动二：模型更复杂 加入了Attention 训练策略改变

在 Test Set 上的表现

指标	更改前	更改后
分类任务类型	5 分类多任务	二分类 (Normal/Abnormal)
测试集准确率 (Acc)	0.7898	0.8623
测试集损失 (Loss)	0.8751	0.2864
M-Score	/	0.8497
测试样本总数	352	6273

改动三：格式转换 、模型压缩

- .pth 权重文件的格式转换成 TF Lite
- 量化 FP 32 原始版本 和 INT 8 版

评估指标	FP32 Version (全精度)	Quant Version (量化版)
模型体积 (Size)	0.30 MB	**0.14 MB**
推理延迟 (Latency)	0.67 ms	1.90 ms （？）
准确率 (Accuracy)	0.8880	0.8840
异常召回率 (Recall)	0.9617	0.9646
M-Score	0.8473	0.8394

部署的文件树

```
FypProj_Deploy/
├── requirements.txt              # 树莓派环境依赖
├── config.yaml                   # 配置文件
├── main_pi.py                    # 部署推理主程序
├── Bluetooth_receiver.py         # 蓝牙接收
├── checkpoints/                  # 模型存放目录
│   ├── best_model.pth            # 原始高精度权重
│   └── heart_model_quant.tflite  # INT8量化模型
└── src/
    ├── model/
    │   └── lightweight_cnn.py    # 模型定义文件
    └── utils/
        └── preprocess_u.py       # 预处理
```

最后的呈现形式（树莓派）

- 按钮若干
- LED若干
  (工作状态显示、诊断结果显示)
- 蜂鸣器（异常时会发出声音）
- I2C OLED显示屏
- 电池模块

进展一：再加入一个模型，用于对于音频质量的评估

Add the **"unsure (too noisy to know)"** category, as originally defined in the PhysioNet 2016 Challenge.
The official dataset provides a reference.csv file regarding audio quality.
Based on the "Acceptable" and "Unacceptable" classifications within this file, a new model has been retrained, designated as Model 0. The previously trained binary classification model for heart sound diseases is referred to as Model 1.

- **The overall logic is as follows:**
  - The audio is first processed by Model 0 to obtain a signal quality assessment.
  - If the assessed quality is poor, the system directly outputs the result as "unsure"
  - If the quality is assessed as good, the audio proceeds to Model 1 for binary heart sound classification. The outputs will be “Normal” or “Abnormal”

进展一：再加入一个模型，用于对于音频质量的评估

Model 0 maintains full consistency with Model 1 in terms of preprocessing pipelines and model architecture.

During the training process, several strategies were implemented to maximize performance:

- **Data Partitioning**: A **Group Split** based on filenames was adopted to prevent data leakage.
- **Class Imbalance:** Used **WeightedRandomSampler** to automatically calculate class weights, forcing the model to encounter minority classes more frequently
- **Optimizer:** Employed **Adam + Weight Decay** to mitigate overfitting.
- **Dynamic Learning Rate Adjustment:** Implemented a scheduler to **halve the learning rate** when the validation loss plateaus, avoiding overfitting.
- **Model Selection:** Saved the model with the **minimum validation loss** rather than the maximum accuracy
- **Early Stopping:** Applied to prevent overfitting and reduce unnecessary training time.

Finally, consistent with Model 1, the model was converted to TFLite format and quantized to INT8 precision for optimized deployment.

进展二：整合功能，移植到树莓派上

After re-flashing the **Raspberry Pi Zero 2W** with a headless version (no desktop), it failed to connect to Wi-Fi for unknown reasons; therefore, I am temporarily using a **Raspberry Pi 4B** instead.

I discovered that connecting to the Raspberry Pi via SSH through VS Code is extremely convenient. This allows me to use VS Code on my computer directly, eliminating the need for vim or nano editors. Additionally, files can be dragged and dropped from local folders directly into the project directory on the Pi.

Since the ESP32-based stethoscope hardware is not currently with me, I have saved a .wav file from the validation set locally for debugging and testing purpose

进展二：整合功能，移植到树莓派上

After configuring the necessary minimal environment, a single inference was performed in Raspberry Pi using a local wav file !

```
(.venv) .venvrasp4b@Rasp4B:~/FypPi $ /home/rasp4b/FypPi/.venv/bin/python /home/rasp4b/FypPi/main_pi_debug.py
🚀 FypProj 双级推理系统 · 最终调试版
============================================================
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
🎵 正在读取音频并提取特征: a0001.wav
📦 预处理成功: 已生成 34 个 2 秒切片
------------------------------------------------------------
片段 01: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 99.73%
片段 02: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 99.91%
片段 03: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 99.99%
片段 04: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 99.41%
片段 05: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 98.66%
片段 06: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 99.88%
片段 07: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 99.30%
片段 08: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 99.09%
片段 09: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 96.70%
片段 10: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 99.20%
片段 11: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 99.88%
片段 12: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 99.73%
片段 13: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 98.27%
片段 14: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 99.13%
片段 15: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 96.88%
片段 16: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 99.53%
片段 17: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 99.93%
片段 18: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 98.80%
片段 19: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 92.13%
片段 20: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 99.75%
片段 21: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 99.77%
片段 22: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 97.35%
片段 23: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 98.81%
片段 24: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 99.49%
片段 25: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 99.54%
片段 26: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 99.80%
片段 27: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 99.82%
片段 28: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 99.70%
片段 29: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 99.97%
片段 30: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 99.98%
片段 31: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 99.98%
片段 32: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 99.66%
片段 33: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 98.39%
片段 34: ✨[诊断通过] 结果: Abnormal (异常) | 置信度: 99.92%
============================================================
✅ 离线验证任务圆满完成。
```

进展一：再加入一个模型，用于对于音频质量的评估

“unsure (too noisy to know)”

reference.csv file regarding audio quality （“Acceptable” and “Unacceptable” ）

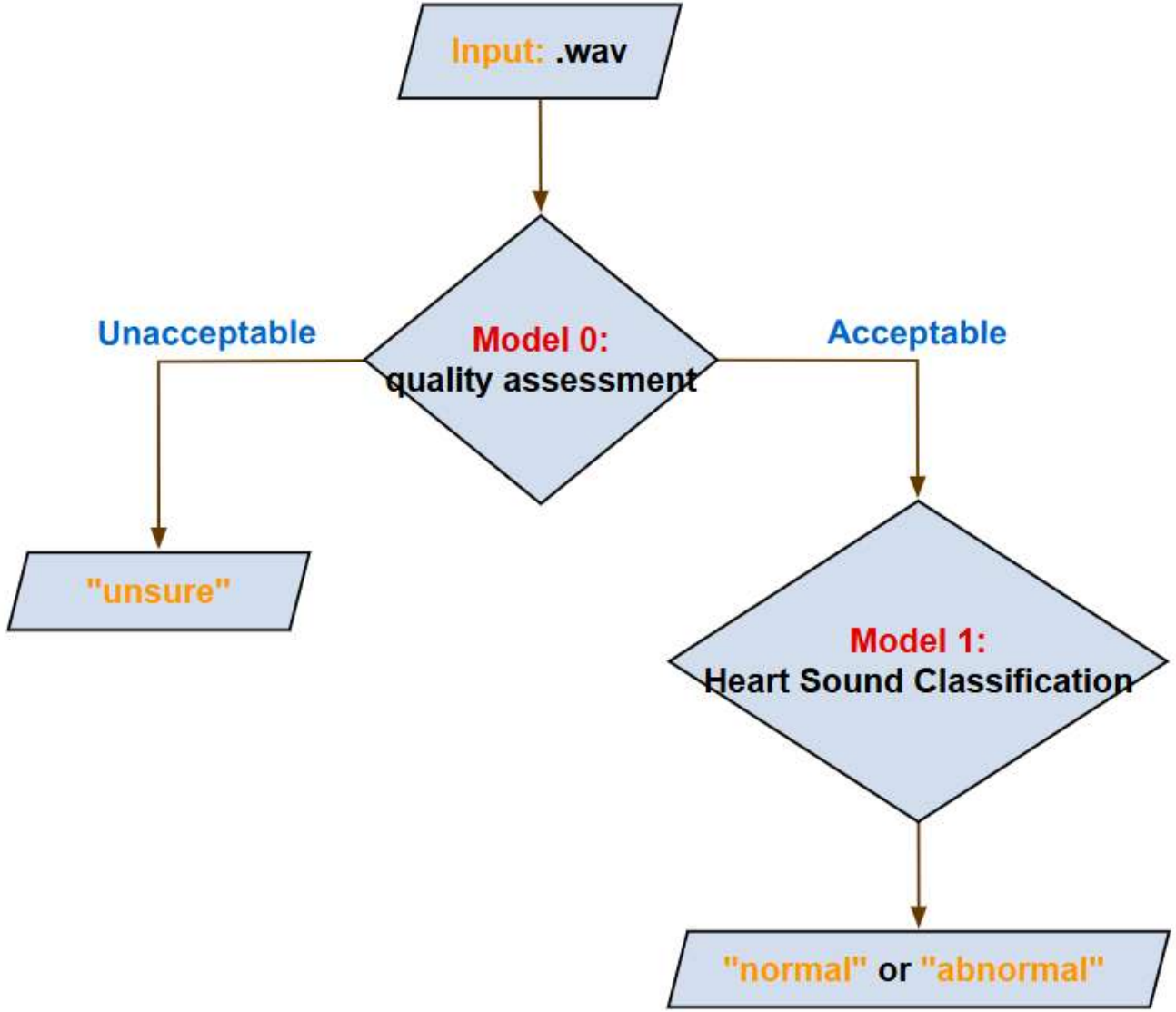
→ Model 0

The only difference with Model 1 is labels.
Preprocess pipeline and model architecture are same.

Label 0 (Unacceptable)	Label 1 (Acceptable)
2876	364

进展一：再加入一个模型，用于对于音频质量的评估

■ The workflow



进展一：再加入一个模型，用于对于音频质量的评估

- **Training Strategy for Model 0:**

Module	Method	Purpose
Data Partitioning	Group Split	Avoid data leakage
Class Imbalance Handling	WeightedRandomSampler	Handle class imbalance
Optimizer	Adam + Weight Decay	Stable and regularized training
Learning Rate Strategy	Dynamic LR Adjustment	Improve convergence
Model Selection	Minimum Validation Loss	Choose best model
Regularization Strategy	Early Stopping	Prevent overfitting

- **Loss Function:** Cross Entropy Loss（交叉熵损失）
- **TFLite format**
- **INT8 precision**

进展二：移植到树莓派上调试

Raspberry Pi Zero 2W ?
✓ Raspberry Pi 4B

■ **Debug version** (without ESP32)

Local: a0001.wav → 34 two second fragment

Execution Time	3.45 second
Initial Memory	131.95 MB
Final Memory	266.76 MB

The following two weeks

- Debug with Esp-32
- Embedded System (switch, led, buzzer, oled display, supply module)
- Support tools

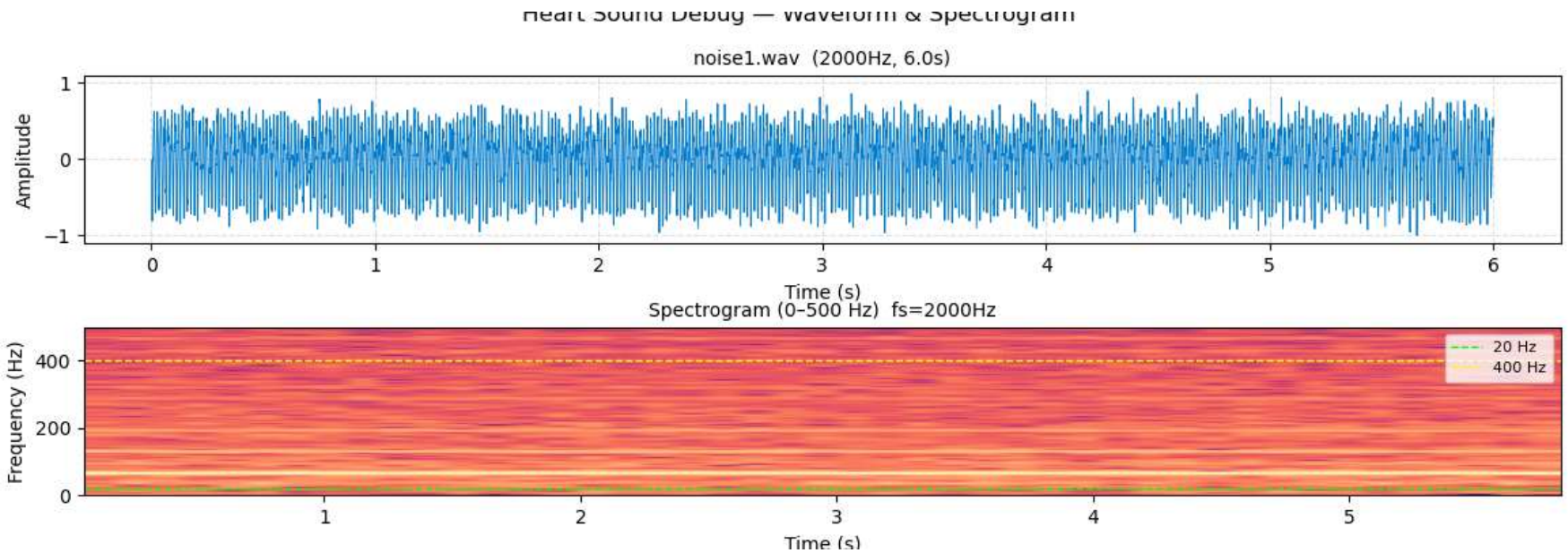
Denoise

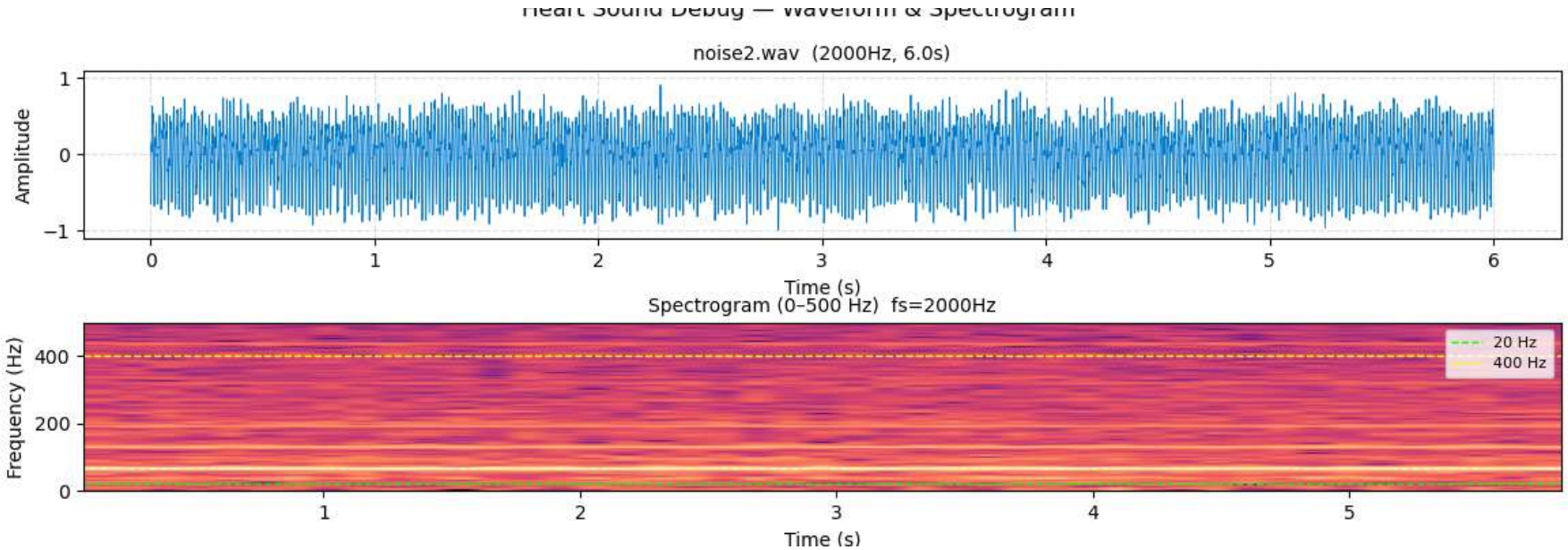
指标	处理后	原始
平均置信度	90.01%	82.10%
最低置信度	72.35% (片段08)	55.55% (片段08)
最高置信度	99.67% (片段10)	99.02% (片段10)
标准差	7.89%	13.42%
置信度<80%的片段数	2	5

BLE 传输

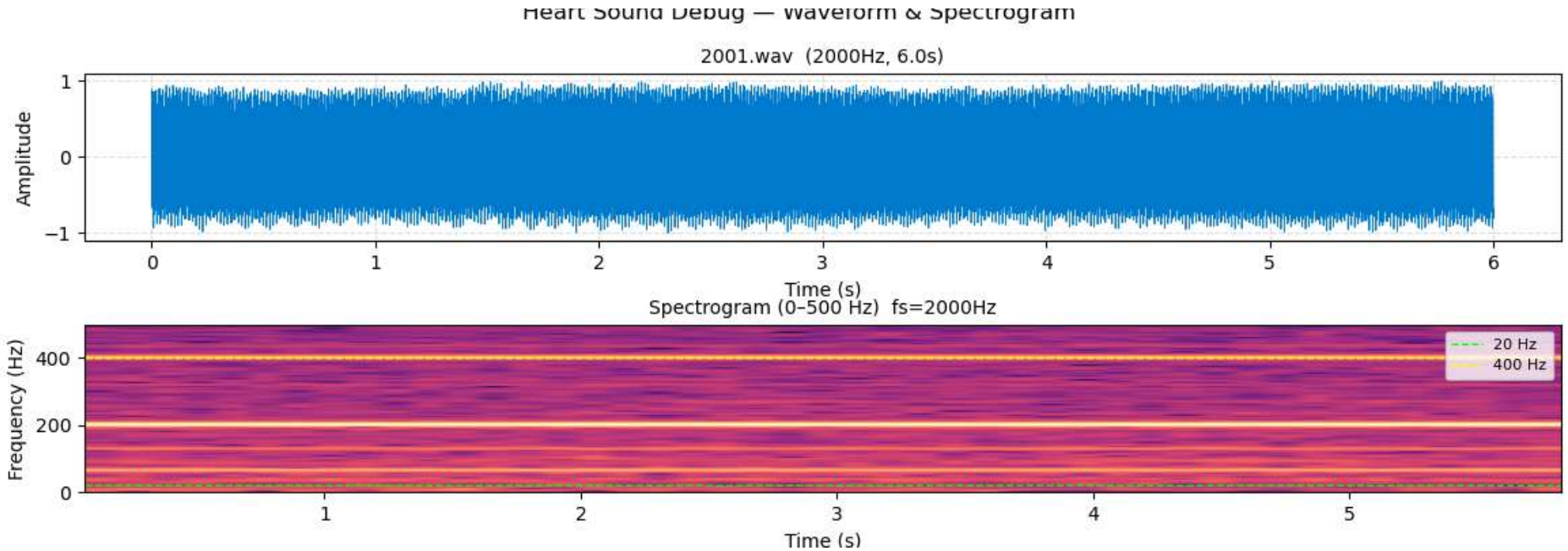
参数	值	说明
单包大小	128 字节	OUT_BLOCK_BYTES
每包样本数	64 samples	128 / 2字节，@ 2000Hz
理论包速率	31.25 包/秒	2000 / 64
理论带宽	3.906 KB/s	31.25 × 128
实测带宽	**3.62 KB/s**	ble_debug 模式3 实测（939包，117.38KB，~32s）
BLE 传输开销	~7%	理论 3.906 KB/s vs 实测 3.62 KB/s

空白环境音

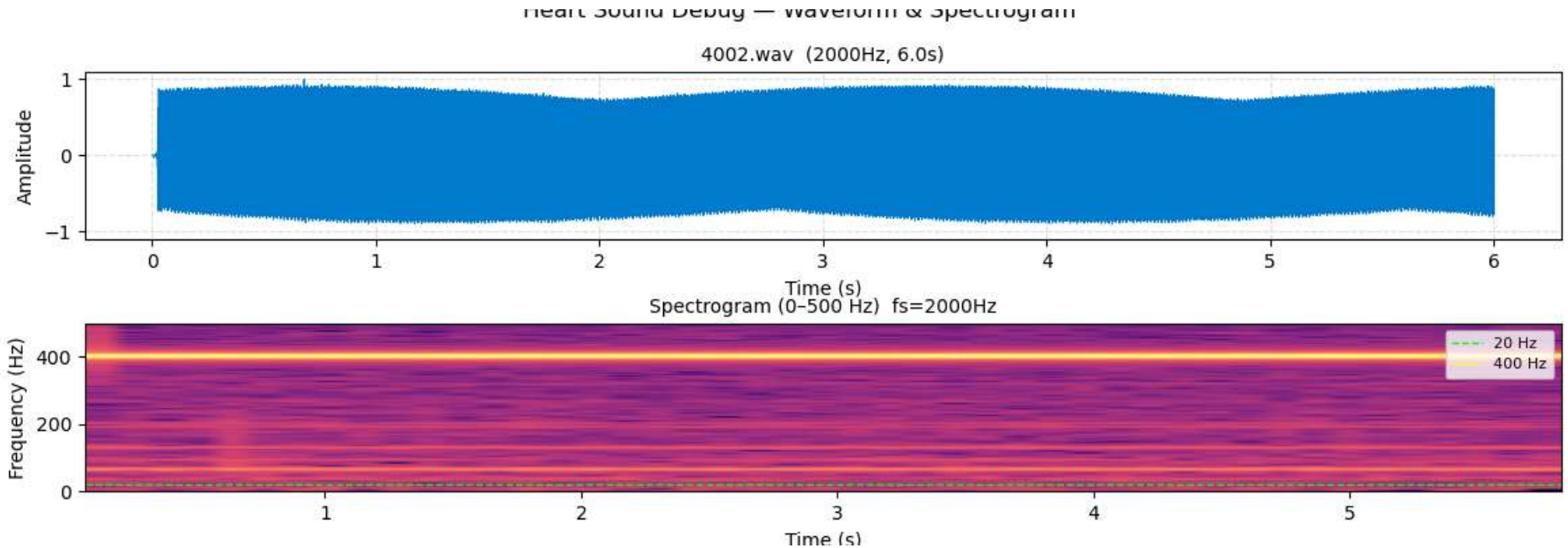

Heart Sound Debug — Waveform & Spectrogram
noise1.wav (2000Hz, 6.0s)
Amplitude
Time (s)
Spectrogram (0–500 Hz) fs=2000Hz
Frequency (Hz)
20 Hz
400 Hz
Time (s)


Heart Sound Debug — Waveform & Spectrogram
noise2.wav (2000Hz, 6.0s)
Amplitude
Time (s)
Spectrogram (0–500 Hz) fs=2000Hz
Frequency (Hz)
20 Hz
400 Hz
Time (s)

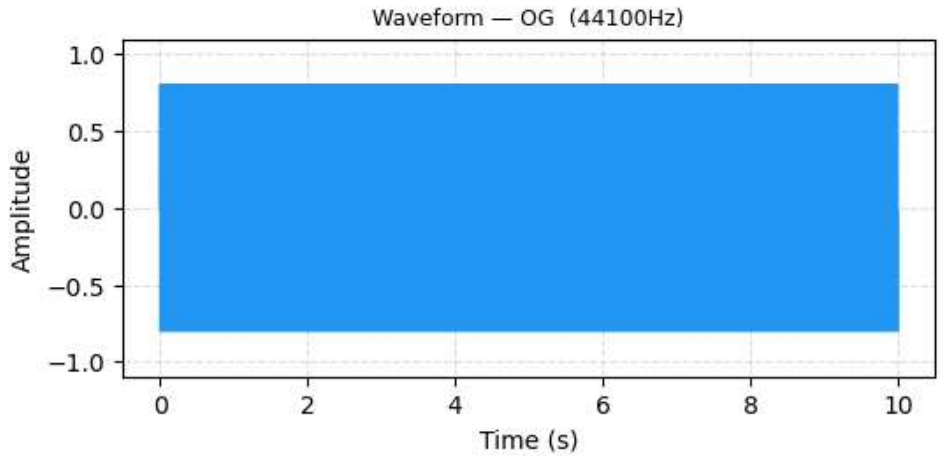
200Hz

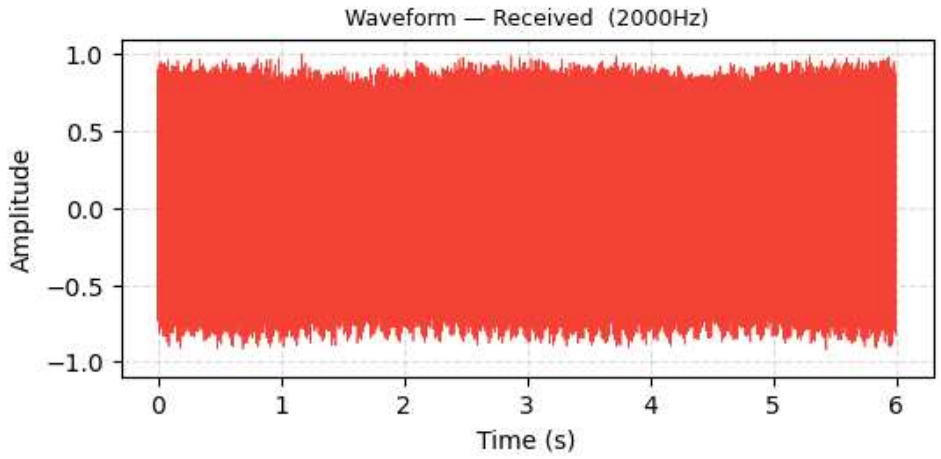

Heart Sound Debug — Waveform & Spectrogram
2001.wav (2000Hz, 6.0s)
Amplitude
Time (s)
Spectrogram (0–500 Hz) fs=2000Hz
Frequency (Hz)
20 Hz
400 Hz
Time (s)

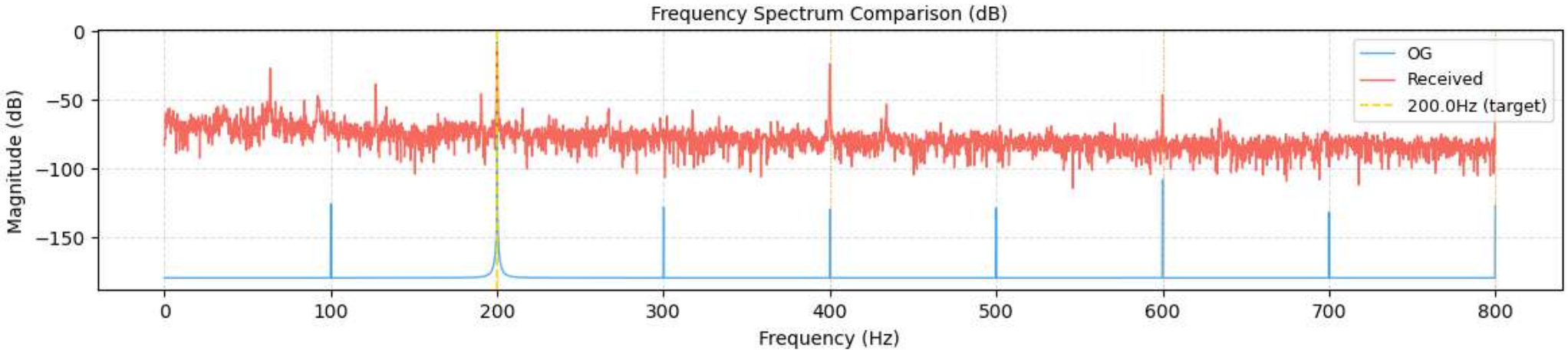
400Hz


Heart Sound Debug — Waveform & Spectrogram
4002.wav (2000Hz, 6.0s)
Amplitude
Time (s)
Spectrogram (0–500 Hz) fs=2000Hz
Frequency (Hz)
20 Hz
400 Hz
Time (s)

200Hz



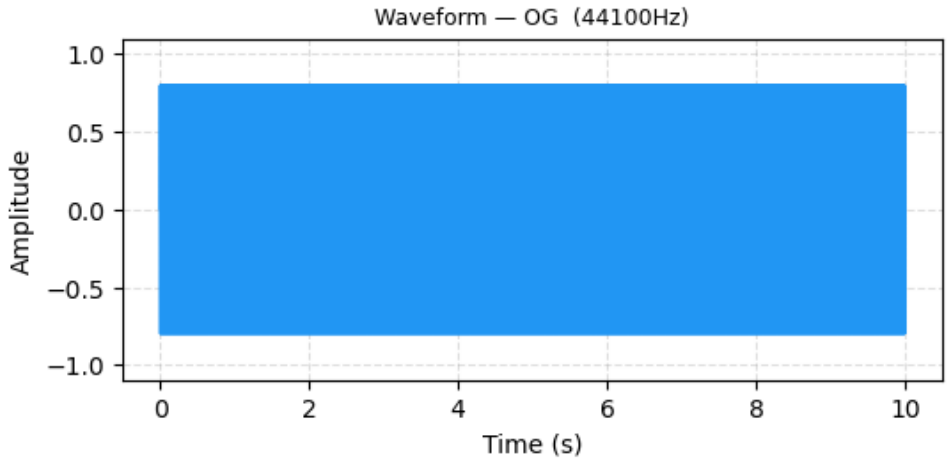


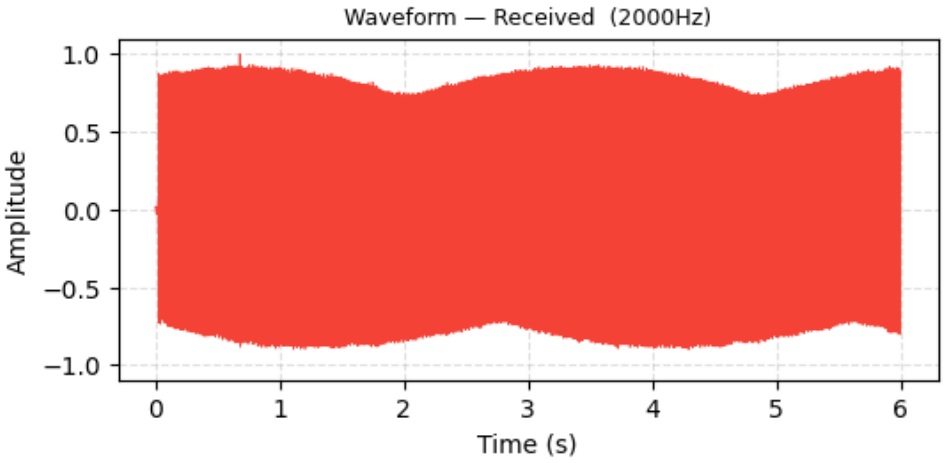


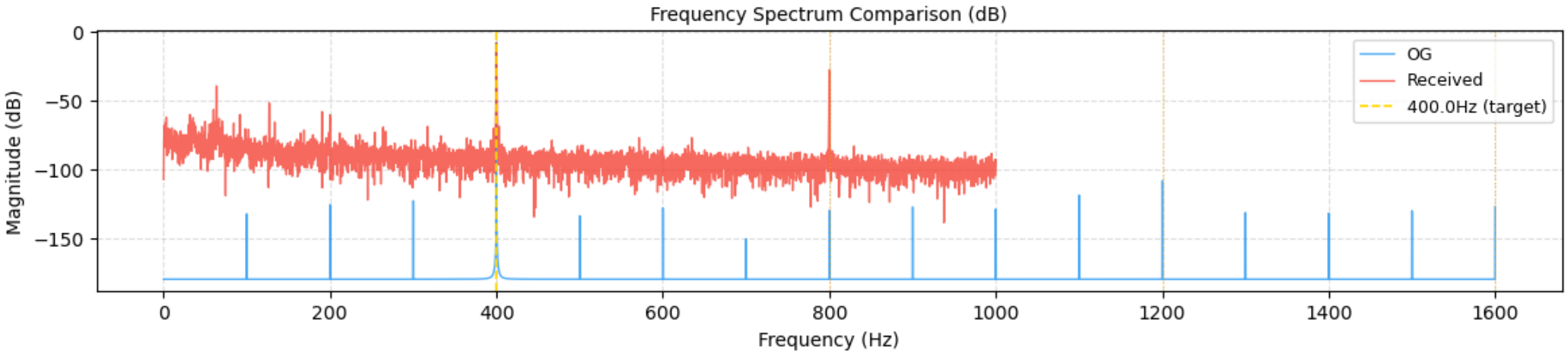
Quantitative Metrics

Metric	OG Signal	Received Signal	Description
Peak Freq	200.00 Hz	200.00 Hz	Target: 200.0 Hz
SNR	95.4 dB	17.4 dB	>20dB acceptable, >40dB good
Purity	100.00%	94.86%	Target freq energy / total energy
THD	0.00%	18.99%	<5% good, >10% distorted
Attenuation	—	-0.8 dB	Negative = loss, ~0 = faithful

400Hz







Quantitative Metrics

Metric	OG Signal	Received Signal	Description
Peak Freq	400.00 Hz	400.00 Hz	Target: 400.0 Hz
SNR	95.4 dB	30.6 dB	>20dB acceptable, >40dB good
Purity	100.00%	98.98%	Target freq energy / total energy
THD	0.00%	9.72%	<5% good, >10% distorted
Attenuation	—	-0.4 dB	Negative = loss, ~0 = faithful

指标	加增益前	加增益后	变化
SNR 200Hz	9-10 dB	16-17 dB	+7 dB ✓
SNR 400Hz	19 dB	30 dB	+11 dB ✓
Purity 200Hz	88-89%	94-95%	+6% ✓
Purity 400Hz	97%	99%	+2% ✓
THD 200Hz	13%	19%	略升
THD 400Hz	12%	9%	略降

推理结果方案

- 每30s接收2s的数据
- 三段2s的音频进行SQA加权平均
- 小于c弃片段

➢ Ex:

三次采样，各自过SQA：

SQA < 0.9 → 直接丢弃，不参与计算

SQA ≥ 0.9 → 参与加权平均

$$\text{final_prob_normal} = \frac{\sum(sqa_i \times prob_normal_i)}{\sum(sqa_i)}$$

假设：

第1次：SQA=0.99，Normal=0.85 → 参与，权重大

第2次：SQA=0.30， → 丢弃

第3次：SQA=0.65，Normal=0.78 → 参与，权重稍小

一条不在数据集中的正常心音

Thr: 80%

运行次序	采集轮次	SQA (Good %)	信号质量判定	诊断结果 (Normal %)	最终状态
第一次	第 1/3 次	96.23%	✅ 合格	**Normal** (51.98%)	**39.03 % Abnormal**
	第 2/3 次	100.00%	✅ 合格	**Abnormal** (8.70%)	
	第 3/3 次	99.99%	✅ 合格	**Normal** (56.90%)	
第二次	第 1/3 次	99.60%	✅ 合格	**Abnormal** (33.77%)	**18.65% Abnormal**
	第 2/3 次	71.87%	⚠️ 质量不足	N/A (跳过)	
	第 3/3 次	99.96%	✅ 合格	**Abnormal** (3.58%)	
第三次	第 1/3 次	76.00%	⚠️ 质量不足	N/A (跳过)	**99.90% Normal**
	第 2/3 次	52.54%	⚠️ 质量不足	N/A (跳过)	
	第 3/3 次	96.64%	✅ 合格	**Normal** (99.90%)	

CPU (主频)	四核 ARM Cortex-A72 (ARM v8) 64位	**1.5 GHz**
GPU	Broadcom VideoCore VI	**500 MHz**

已经改成实时的流式推理
20s一个chunk

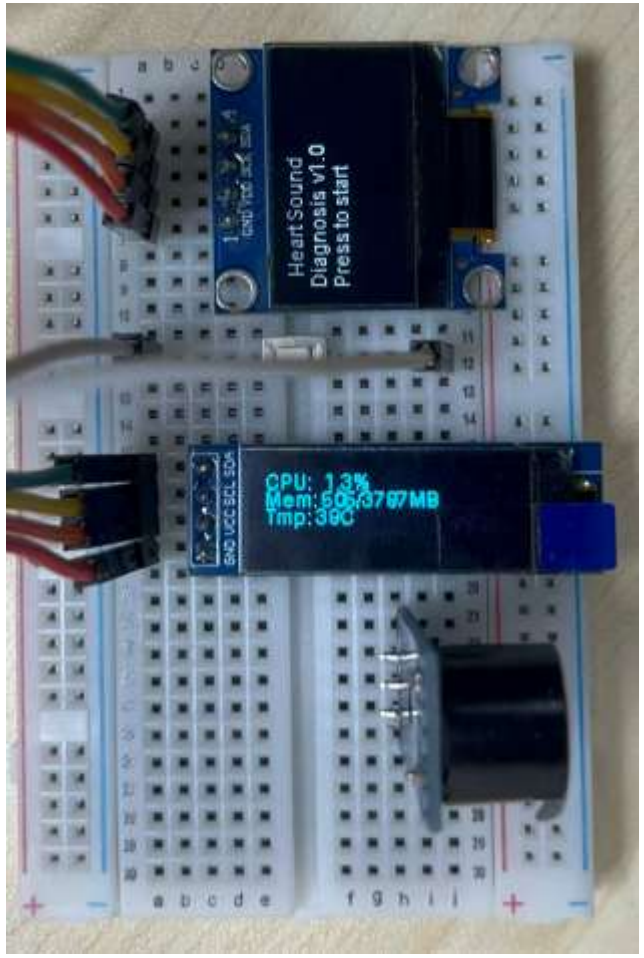
```
(.venv) rasp4b@Rasp4B:~/FypPi $ python main_pi.py
INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
正在连接 ESP32: 80:F1:B2:ED:B4:12...
已连接，MTU: 517 字节
流式采集启动（每 20s 一块，滑窗 2.0s / hop 1.0s）
Ctrl+C 停止
```

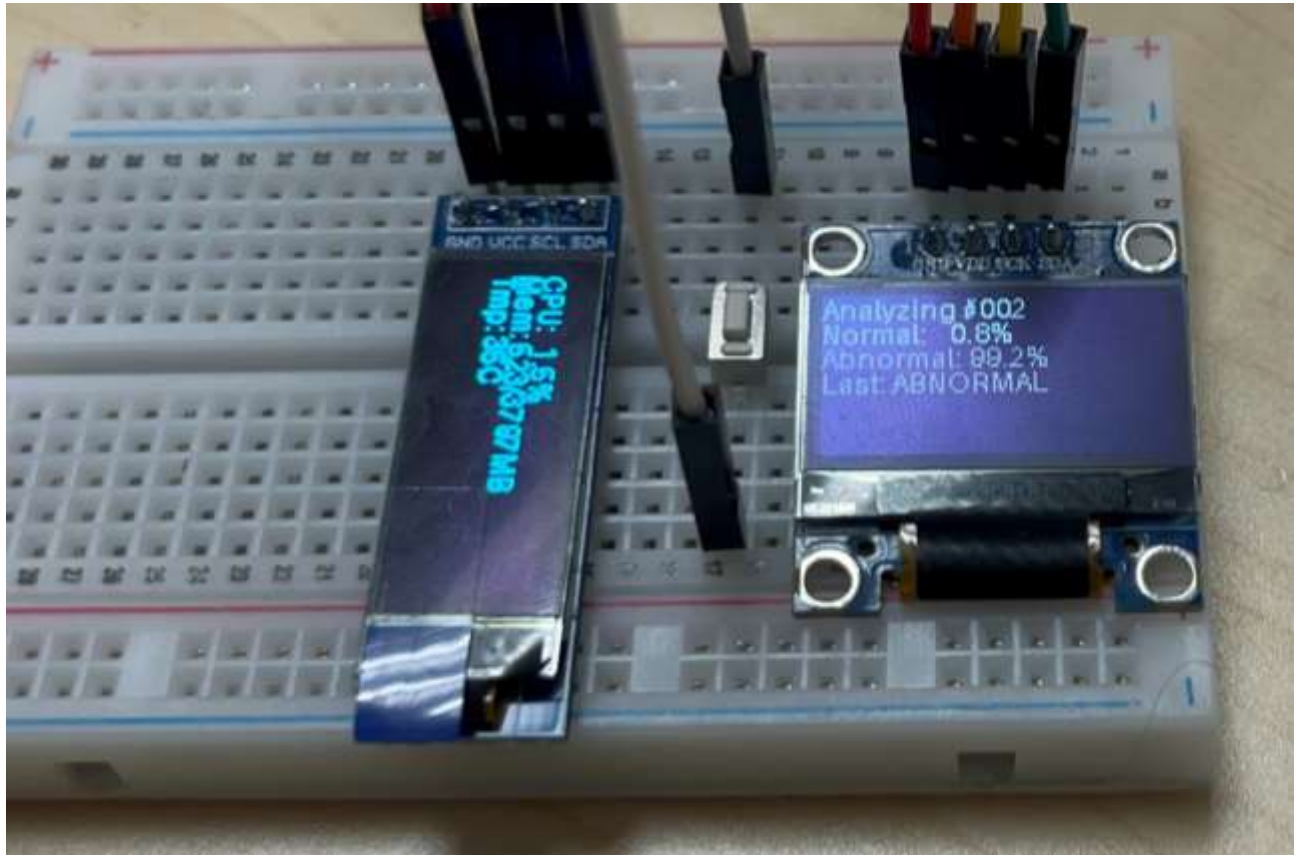
```
[块 001] 推理中（20s / 19 窗口）...
  w01:0.99✓  w02:1.00✓  w03:0.99✓  w04:0.96✓  w05:0.99✓  w06:1.00✓  w07:1.00✓  w08:1.00✓
  w09:0.99✓  w10:1.00✓  w11:0.99✓  w12:0.99✓  w13:0.89✓  w14:1.00✓  w15:0.99✓  w16:0.99✓
  w17:0.96✓  w18:0.97✓  w19:0.94✓
[块 001]  Abnormal   prob_normal=2.16%  (19/19 窗口有效)
[块 001]  已保存: /home/rasp4b/FypPi/debug_records/abnormal_c0001_1774965790.wav
```

```
[块 002] 推理中（20s / 19 窗口）...
  w01:0.00✗  w02:0.00✗  w03:0.00✗  w04:0.02✗  w05:0.94✓  w06:0.99✓  w07:0.11✗  w08:1.00✓
  w09:1.00✓  w10:1.00✓  w11:1.00✓  w12:1.00✓  w13:1.00✓  w14:0.85✓  w15:0.90✓  w16:0.98✓
  w17:0.40✗  w18:0.70✓  w19:1.00✓
[块 002]  Abnormal   prob_normal=3.44%  (13/19 窗口有效)
[块 002]  已保存: /home/rasp4b/FypPi/debug_records/abnormal_c0002_1774965802.wav
```

系统设计：2块OLED屏幕 和 1个按键

(蜂鸣器待调试 三色LED灯和电池模块在路上)





- 2块OLED屏幕，因为地址冲突，走了两条不同的I2C通路
- 按键有配置软件消抖20ms

系统设计：全流程

系统设计：数据管理

模块一：Structured Logs

- **存储逻辑**：按日期分目录管理 (records/YYYY-MM-DD/)。
- **记录维度**：实时记录SQA 质量评分与诊断概率。
- **文件格式**：CSV 格式

模块二：Abnormal Archive

- **触发机制**：检测到 Abnormal 结果时自动激活。
- **处理流程**：原始 PCM 流并转码为 WAV 格式。

模块三：JSONL Summary

- **核心功能**：单行 JSON 记录 (summary.jsonl)
- **数据闭环**：包含时间戳、置信度、有效片段数及 Noise 信号标记。

模块四：Auto-Cleanup

- **过期策略**：本地数据循环滚动，默认保留最近 30 天。

系统设计

- **Main_pi.py 开机自启（systemd）**

- **Watchdoor dog**
- 主循环每 30s 写一次心跳时间戳到 `/tmp/heartbeat.ts`
- 独立看门狗进程检查时间戳，超时 90s 则重启主服务

- **日志管理**
- 使用 Python `logging` 模块，级别 INFO，文件 `logs/app.log`